

ASIC Design Flow

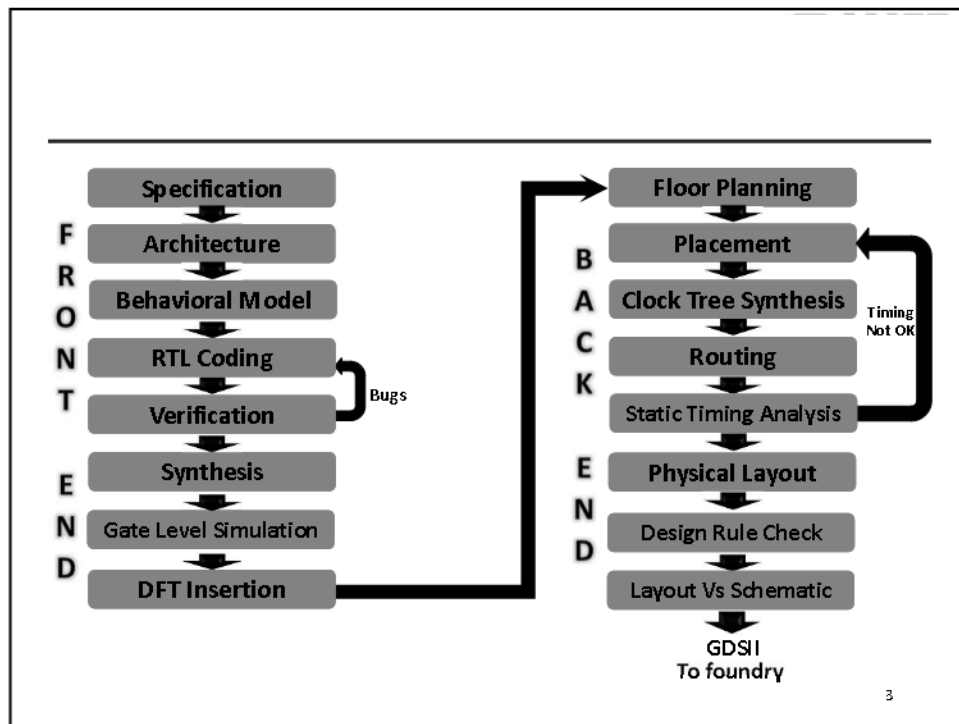
1

1

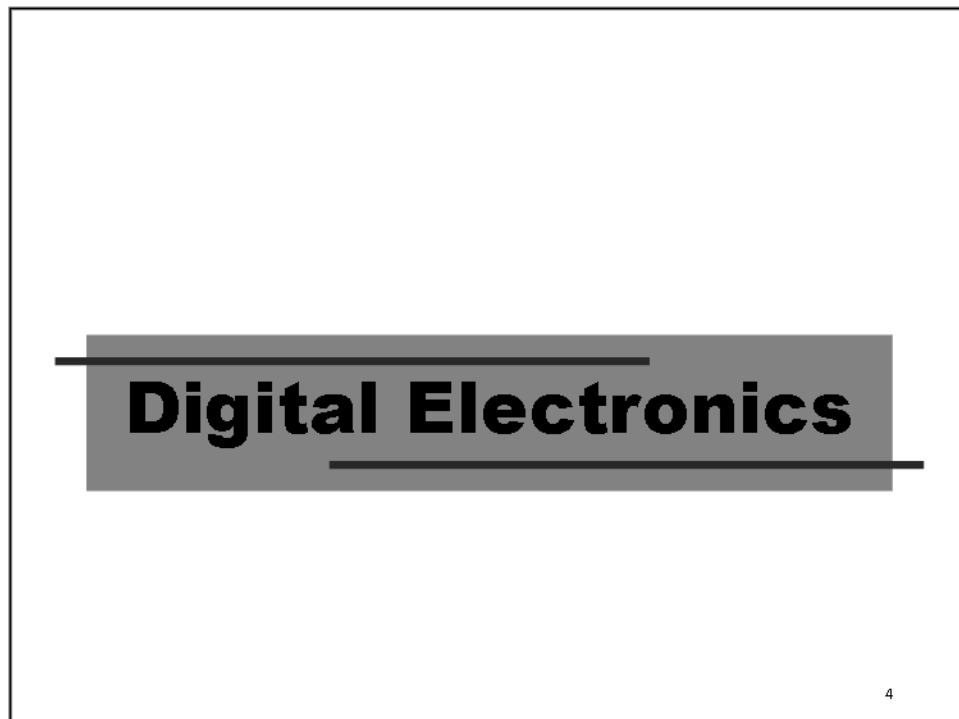
2

2

1



3



4

Course Agenda

- ♦ **Introduction**
- ♦ **Number Systems & Codes**
 - ♦ Conversion Techniques
 - ♦ Signed Numbers
 - ♦ Arithmetic Operations
 - ♦ Codes
- ♦ **Logic Circuits & Boolean Algebra**
 - ♦ Logic Gates
 - ♦ Boolean Algebra
 - ♦ NAND & NOR Circuits
 - ♦ Logic Minimization & K-Map
 - ♦ Don't care Conditions
- ♦ **Combinational Logic Circuits**
 - ♦ Design Procedure
 - ♦ Delays in combinational circuits
 - ♦ Adders & Subtractors
 - ♦ Encoders & Decoders
 - ♦ Multiplexer & Demultiplexer.
 - ♦ Implementation of Boolean functions using Multiplexers.
 - ♦ Tristate Buffers.

5

5

Course Agenda

- ♦ **Sequential Logic Circuits**
 - ♦ Latches
 - ♦ Flipflops
 - ♦ Delays in Sequential Circuits
 - ♦ Registers
 - ♦ *Buffer & Shift Registers*
 - ♦ Counters
 - ♦ *Classification*
 - ♦ *Modulus of a counter*
 - ♦ *Counter Design Process*
 - ♦ *Frequency division*
 - ♦ *Shift register Counters*
- ♦ **Finite State Machines**
 - ♦ Mealy Machine
 - ♦ Moore Machine
- ♦ **Glitches & Hazards**
 - ♦ Definition
 - ♦ Elimination of Hazards
- ♦ **Memories**
 - ♦ Classification of memories.
 - ♦ Basic Concepts of Memories
 - ♦ Composition of memories.

6

6

3

Number Systems

7

7

Agenda

- ♦ Binary, Decimal, Octal and Hexa Decimal Number
- ♦ Number system Conversions
- ♦ Signed Numbers
- ♦ Sign Magnitude Representation
- ♦ Complement Representation
- ♦ Arithmetic Operations using Complement number system.
- ♦ Codes

8

8

4

Number Systems

What is Number System?

- ♦ It can be defined as the Language of Numbers
- ♦ Each Language contains different set of symbols for representation of numbers.
- ♦ The most commonly used number systems are
 - ♦ Decimal Number System.
 - ♦ Binary Number System.
 - ♦ Octal Number System.
 - ♦ Hexa Decimal Number System.

Radix or Base

9

9

Number Systems

- ♦ Decimal Number System

{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

Radix - 10

Example : 12495₍₁₀₎

- ♦ Binary Number System

{ 0, 1 }

Radix - 2

Example : 1111101₍₂₎

- ♦ Octal Number System

{ 0, 1, 2, 3, 4, 5, 6, 7 }

Radix - 8

Example : 71245₍₈₎

- ♦ Hexa Decimal Number System

{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }

16

9ABD5₍₁₆₎

10

10

5

Number system conversions

Any radix to Decimal number system

No. of digits to the left of the radix point.

$$D = \sum_{i=-n}^{p-1} (d_i * r^i)$$

Value of the number

Radix of the number system

No. of digits to the right of the radix point.

11

11

Binary to Decimal Conversion

Example : $\overset{d_3 d_2}{1010}.\overset{d_1 d_0}{01}_{(2)}$

In this case,

$$D = \sum_{i=-n}^{p-1} (d_i * r^i) \Rightarrow D = \sum_{i=-2}^{4-1=3} (d_i * 2^i)$$

p = No. of digits to the left of the radix point = 4

n = No. of digits to the right of the radix point = 2

d = Value of the number

r = Radix of the number system = 2

$$\begin{aligned} D &= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) \\ &= 0.25 + 0 + 0 + 2 + 0 + 8 \\ &= 10.25_{(10)} \end{aligned}$$

12

12

Decimal to other number system

Integer part of Decimal number

Successive Division
by Required Radix number

Fractional part of Decimal number

Successive Multiplication
by Required Radix number

13

13

Decimal to other number system

Example : $125_{(10)} = ?_{(2)}$

Successive division

2	125	
2	62	- 1
2	31	- 0
2	15	- 1
2	7	- 1
2	3	- 1
	1	- 1

$$125_{(10)} = 1111101_{(2)}$$

14

14

Decimal to other number system

Example : $0.7_{(10)} = ?_{(2)}$

♦ Successive multiplication

$0.7 * 2 = 1.4$	$\Rightarrow 1$
$0.4 * 2 = 0.8$	$\Rightarrow 0$
$0.8 * 2 = 1.6$	$\Rightarrow 1$
$0.6 * 2 = 1.2$	$\Rightarrow 1$
$0.2 * 2 = 0.4$	$\Rightarrow 0$
$0.4 * 2 = 0.8$	$\Rightarrow 0$

$$0.7_{(10)} = 0.101100_{(2)}$$

$$125.7_{(10)} = 111101.101100_{(2)}$$

15

15

Binary to Octal

$(11101.011)_2 = (?)_8$

Step 1 : Create groups with 3 no. of binary digits from the fraction point.

11101.011

$\underbrace{011}_3$ $\underbrace{101}_5$ $\underbrace{011}_3$

Appending '0's' at MSB position of integer part, doesn't change the value of the number.

Step 2 : Represent each group with an equivalent octal number.

$$\Rightarrow (11101.011)_2 = (35.3)_8$$

16

16

Binary to Hexa

$$(11101.011)_2 = (?)_{16}$$

Step 1 : Create groups with 4 no. of binary digits from the fraction point.

$$\begin{array}{ccc} & & 11101.011 \\ \underbrace{0001} & \underbrace{1101} & \underbrace{0110} \\ 1 & D & 6 \end{array}$$

Appending '0's at MSB position of integer part & Appending '0's at LSB position in the fraction part of the number, doesn't change the value of the number.

Step 2 : Represent each group with an equivalent Hexa number.

$$\Rightarrow (11101.011)_2 = (1D.6)_{16}$$

17

17

Octal to Binary

$$(476.543)_8 = (?)_2$$

Step 1 : Represent each digit with a 3-bit binary equivalent

$$\begin{array}{ccccccc} \underbrace{4} & \underbrace{7} & \underbrace{6} & . & \underbrace{5} & \underbrace{4} & \underbrace{3} \\ 100 & 111 & 110 & & 101 & 100 & 011 \end{array}$$

$$\Rightarrow (476.543)_8 = (100\ 111\ 110.101\ 100\ 011)_2$$

18

18

Hexa to Binary

$$(ADE.54C)_{16} = (?)_2$$

Step 1 : Represent each digit with a 4-bit binary equivalent

$$\begin{array}{ccccccc} \underbrace{A} & \underbrace{D} & \underbrace{E} & . & \underbrace{5} & \underbrace{4} & \underbrace{C} \\ 1010 & 1101 & 1110 & & 0101 & 0100 & 1100 \end{array}$$

$$\Rightarrow (ADE.54C)_{16} = (1010\ 1101\ 1110.0101\ 0100\ 1100)_2$$

19

19

Signed Numbers

- ♦ Schemes to represent the signed numbers
 - Sign – Magnitude Representation
 - Complement Representation

20

20

10

Sign – Magnitude Representation

- ♦ MSB is used as sign bit & the other bits represents the magnitude.

0 – for Positive
1 – for Negative

- ♦ Example:

$1111_{(2)}$	=	15	⇒	Unsigned Number
$\underline{0}111_{(2)}$	=	+15	⇒	Signed <u>Positive</u> Number
$\underline{1}111_{(2)}$	=	-15	⇒	Signed <u>Negative</u> Number

- ♦ Range of the n bit signed magnitude integer :

$-(2^{(n-1)} - 1)$ to $+(2^{(n-1)} - 1)$

- ♦ Shortcomings:

- Complex Circuits
 - Two representations for 0
- 00000 +0
10000 -0

21

21

Complement Representation

- ♦ 1's complement - Complementing all bits

- ♦ Example :

$1111_{(2)}$ = +15₍₁₀₎
 $0000_{(2)}$ = -15₍₁₀₎

- ♦ 2's complement - Add 1 to 1's complement.

- ♦ Example :

$1111_{(2)}$ = +15₍₁₀₎
 $0001_{(2)}$ = -15₍₁₀₎

- ♦ Easy Method to get 2's Complement

$1011010_{(2)}$
 $0100101_{(2)}$ ⇒ 1's Complement
+1
 $0100110_{(2)}$ ⇒ 2's Complement

$1011010_{(2)}$
 $0100110_{(2)}$ ⇒ 2's Complement

Start from LSB and use the same digits until, you come across '1' and complement the remaining bits.

22

22

11

2's Complement for Binary Arithmetic

Addition

7	⇒	0 1 1 1
+4	⇒	+ 0 1 0 0
11	⇔	0 1 0 1 1

Subtraction

7	7	⇒	0 1 1 1
-4	+ (-4)	⇒	+ 1 1 0 0
3	⇔	1 0 0 1 1	X

Key Points

- ♦ For Subtraction, Take 2's Complement of the negative number and add it with the other number.
- ♦ Carry 1 indicates positive number
- ♦ Carry 0 indicates negative number.

4	4	⇒	0 1 0 0
-7	+ (-7)	⇒	+ 1 0 0 1
-3	⇔	1 1 0 1	

2's complement of 1101 is 0011

23

23

Complement Number system

System	Base	Possible complement systems
Binary	2	1's comp & 2's comp
Decimal	10	9's comp & 10's comp
Octal	8	7's comp & 8's comp
Hexa	16	15's comp & 16's comp
Unknown	n	(n-1)'s comp & n's comp

- ✓ n's complement of a number can be obtained by adding 1 to the (n-1)'s complement.
- ✓ (n-1)'s complement of a 'p' digit number can be obtained by subtracting the number from the maximum 'p' digit number.

24

24

12

Arithmetic using Complement System

Example :

$34_{(8)} - 27_{(8)}$
8's comp of $27_{(8)}$ is $(77 - 27) + 1 = 51$

34	$\Rightarrow$	34
-27	$\Rightarrow$	$+51$
$05_{(8)}$	$\Leftrightarrow$	105

'77' is the maximum 2 digit octal number

Additional Carry (if any) has to be discarded.

25

25

Arithmetic using Complement System

Example :

$2A_{(16)} - 4C_{(16)}$
16's comp of $4C_{(16)}$ is $(FF - 4C) + 1 = B4$

$2A$	$\Rightarrow$	$2A$
$-4C$	$\Rightarrow$	$+B4$
		$0DE$

'FF' is the maximum 2 digit hexa number

If the additional carry is not generated, It indicates that the result is a negative number.

16's comp of $DE_{(16)}$ is $(FF - DE) + 1 = 22$
So, $2A_{(16)} - 4C_{(16)} = -22_{(16)}$

To get the magnitude of result, 16's complement has to be calculated for the sum obtained.

26

26

13

Codes

- ♦ Symbolic representation
- ♦ Types of codes
 - ♦ Weighted binary codes.
 - ♦ Non-weighted codes.
 - ♦ Alphanumeric codes.
 - ♦ Error detecting codes.

27

27

Weighted Binary Codes

Decimal	8421	5421	2421
0	0000	0000	0000
1	0001	0001	0001
2	0010	0010	0010
3	0011	0011	0011
4	0100	0100	0100
5	0101	1000	1011
6	0110	1001	1100
7	0111	1010	1101
8	1000	1011	1110
9	1001	1100	1111

BCD Code

BCD code for $874_{(10)}$
 $874 = 1000\ 0111\ 0100_{(BCD)}$

28

28

14

Non – Weighted Codes

- ♦ Non –weighted codes are not positional weighted.
- ♦ Example :

Excess – 3 Code, Gray Code

Gray Code

- The no. of bit changes is only one between two consecutive numbers
- Used in low power applications

Decimal	Binary	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101

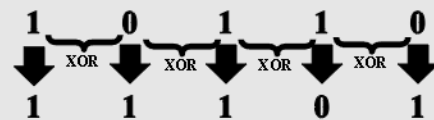
29

29

Binary and Gray code Conversion

- ♦ Binary to Gray code Conversion

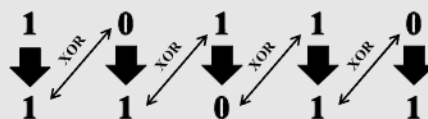
Convert $[10110]_2$ to gray code



$$10110_{(2)} = 11101_{(\text{gray})}$$

- ♦ Gray to Binary code Conversion

Convert $[10110]_{\text{gray}}$ to Binary code



$$10110_{(\text{gray})} = 11011_{(2)}$$

30

30

15

ASCII

- ♦ ASCII stands for American Standard Code for Information Interchange

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	EF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	--	#	\$	%	&	.	{	}	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	:	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	/	]	^	_
6	.	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

- ♦ The code uses 7 bits to encode 128 unique characters

31

31

Other codes

- ♦ Parity codes - Parity bits for detection of errors.
- ♦ Hamming code is for correcting the errors.
- ♦ Hollerith code.
- ♦ EBCDIC (Extended Binary Coded Decimal Interchange Code)
- ♦ ASCII Redefined with 8 bits to represent 256 different characters.

32

32

16

Logic Circuits

33

33

Agenda

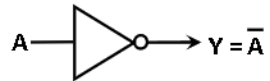
- ◆ Logic gates
- ◆ Boolean Algebra
- ◆ NAND and NOR Circuits
- ◆ Logic Minimisation

34

34

Logic Gates

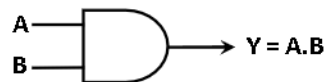
NOT gate (Inverter)



Truth Table of NOT gate

A(I/P)	Y(O/P)
0	1
1	0

AND gate



Truth Table of AND gate

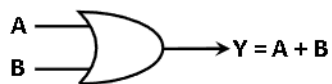
A	B	Y(O/P)
0	0	0
0	1	0
1	0	0
1	1	1

35

35

Logic Gates

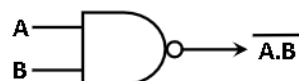
OR gate



Truth Table of OR gate

A	B	Y(O/P)
0	0	0
0	1	1
1	0	1
1	1	1

NAND gate



Truth Table of NAND gate

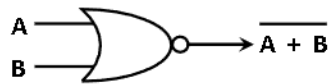
A	B	Y(O/P)
0	0	1
0	1	1
1	0	1
1	1	0

36

36

Logic Gates

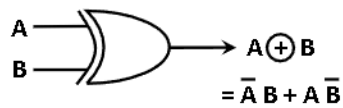
NOR gate



Truth Table of NOR gate

A	B	Y(O/P)
0	0	1
0	1	0
1	0	0
1	1	0

XOR gate



Truth Table of XOR gate

A	B	Y(O/P)
0	0	0
0	1	1
1	0	1
1	1	0

...

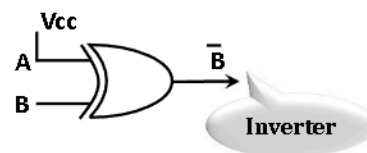
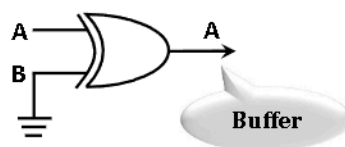
37

37

Properties of XOR Gate

A	B	EXOR
0	0	0
0	1	1
1	0	1
1	1	0

- ✓ if one of the inputs is '0', then the output is same as the second input.
- ✓ If one of the inputs is '1', then, the output is the complement of the second input.



38

38

19

Properties of XOR Gate

A	B	EXOR
0	0	0
0	1	1
1	0	1
1	1	0

- ✓ Odd parity detector.
- ✓ Even parity generator.

Since, the total no. of 1's in inputs and output is even. So, we can consider it as Even Parity Generator.

Since, No. of 1's in inputs, is odd. Output has become 1. So, we can consider it as odd parity detector.

39

39

Logic Gates

EXNOR gate



Can be called
as Equality
Gate

Truth Table of EXNOR gate

A	B	Y(O/P)
0	0	1
0	1	0
1	0	0
1	1	1

Output is complement of EXOR gate.

40

40

20

Boolean Algebra

$$(T1) \quad X + 0 = X$$

$$(T2) \quad X + 1 = 1$$

$$(T3) \quad X + X = X$$

$$(T4) \quad (X')' = X$$

$$(T5) \quad X + X' = 1$$

$$(T1') \quad X \cdot 1 = X$$

$$(T2') \quad X \cdot 0 = 0$$

$$(T3') \quad X \cdot X = X$$

$$(T5') \quad X \cdot X' = 0$$

$$(T6) \quad X + Y = Y + X$$

$$(T7) \quad X + (Y + Z) = (X + Y) + Z$$

$$(T8) \quad X \cdot Y + X \cdot Z = X \cdot (Y + Z)$$

$$(T9) \quad X + X \cdot Y = X$$

$$(T10) \quad X \cdot Y + X \cdot Y' = X$$

$$(T6') \quad X \cdot Y = Y \cdot X;$$

$$(T7') \quad X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$$

$$(T8') \quad (X + Y) \cdot (X + Z) = X + (Y \cdot Z)$$

$$(T9') \quad X \cdot (X + Y) = X$$

$$(T10') \quad (X + Y) \cdot (X + Y') = X$$

$$(T11) \quad X \cdot Y + Y \cdot Z + X' \cdot Z = X \cdot Y + X' \cdot Z$$

$$(T11') \quad (X + Y) \cdot (Y + Z) \cdot (X' + Z) = (X + Y) \cdot (X' + Z)$$

41

41

Boolean Algebra

Expression

Dual of the Expression

$$A + B = B + A$$

$$A + AB = A$$

$$A + \bar{A}B = A + B$$

$$A \cdot B = B \cdot A$$

$$A \cdot (A + B) = A$$

$$A \cdot (\bar{A} + B) = AB$$

Famous De Morgan Rules

Dual of the Expression

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

Consensus Theorem

Dual of the Expression

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

$$(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$$

42

42

21

Proof for Consensus Theorem

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

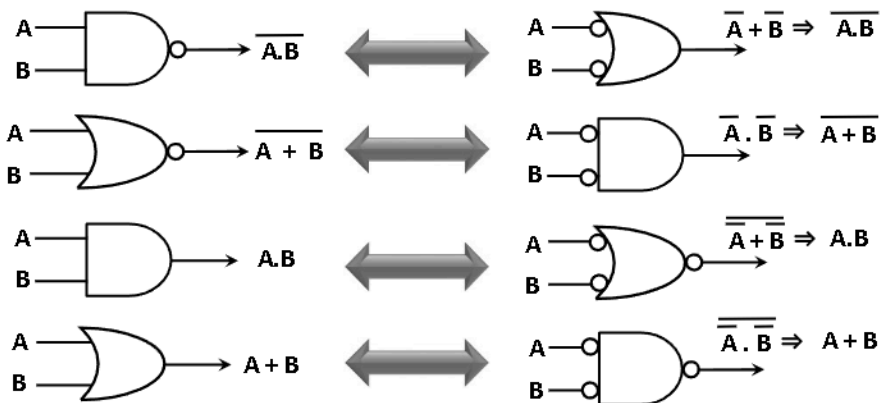
$$\begin{aligned} & AB + \bar{A}C + BC \\ &= AB + \bar{A}C + (A + \bar{A})BC \\ &= AB + \bar{A}C + ABC + \bar{A}BC \\ &= AB(1 + C) + \bar{A}C(1 + B) \\ &= AB + \bar{A}C \end{aligned}$$

43

43

Gate Equivalents

De Morgan's rules.



44

44

22

Complement

- ♦ The complement of a function F is denoted by F' or $\bar{F}$
- ♦ DeMorgan's theorems to compute the complement of a function.

Another way of finding Complement function

$$F(A,B,C) = A\bar{B} + B\bar{D} + \bar{A}\bar{D}$$

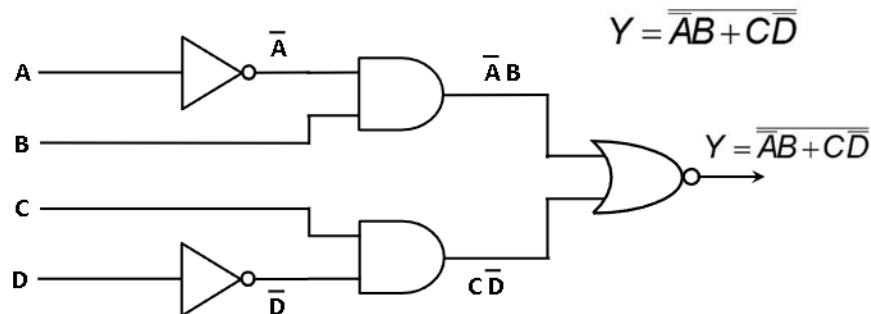
$$\begin{aligned} F(A,B,C) &= (A\bar{B}) + (B\bar{D}) + (\bar{A}\bar{D}) \\ \text{Dual} = F_d &= (A + \bar{B})(B + \bar{D})(\bar{A} + \bar{D}) \\ \text{Comp} = \bar{F} &= (\bar{A} + \bar{B})(\bar{B} + \bar{D})(\bar{A} + \bar{D}) \\ &= \bar{F} = (\bar{A} + B)(\bar{B} + D)(A + D) \end{aligned}$$

45

45

Logic Circuit from function

Implement the function using Logic Gates.



✓ Consider each sum or product term individually and try to obtain them separately by using appropriate logic gates.

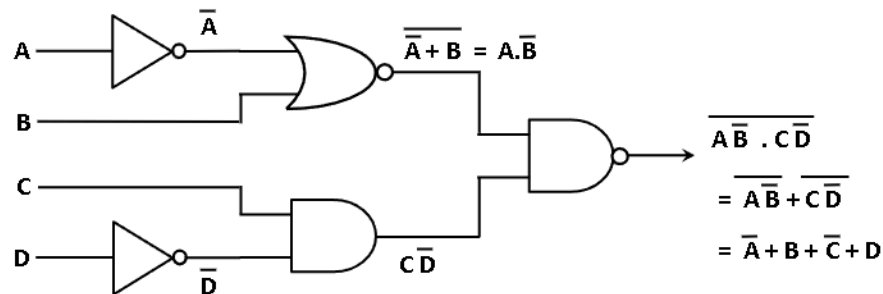
46

46

23

Function from circuit

Determine the output function from the given circuit.



- ✓ Work progressively from the inputs to the outputs.
- ✓ Derive the logic expressions for the outputs of all the gates.

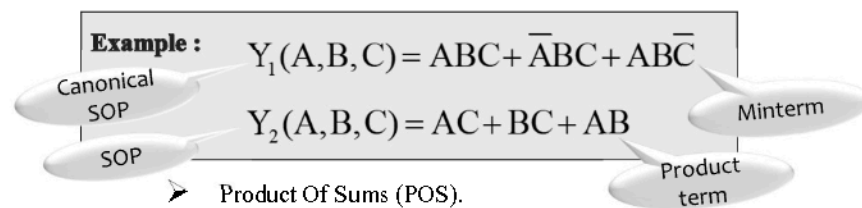
47

47

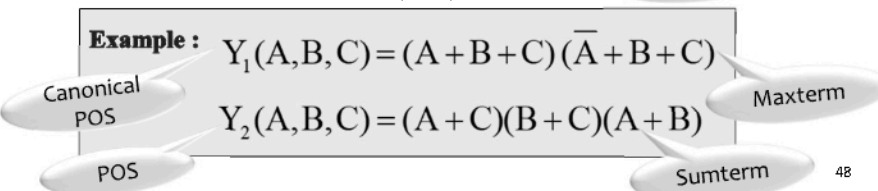
SOP & POS

♦ Any Boolean function can be expressed in two different standard forms.

➤ Sum Of Products (SOP),



➤ Product Of Sums (POS),



48

48

24

SOP

A	B	C	Y(O/P)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$\Rightarrow (\overline{A}BC)$$

$$\Rightarrow (\overline{A}BC)$$

$$\Rightarrow (\overline{A}BC)$$

$$Y = (\overline{A}BC) + (\overline{A}BC) + (\overline{A}BC)$$

$$Y(A, B, C) = \sum m(2, 3, 6)$$

To get Complement, Write only the left out Terms

$$\overline{Y}(A, B, C) = \sum m(0, 1, 4, 5, 7)$$

49

49

POS

A	B	C	Y(O/P)
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

$$\Rightarrow (A + \overline{B} + C)$$

$$\Rightarrow (A + \overline{B} + \overline{C})$$

$$\Rightarrow (\overline{A} + \overline{B} + C)$$

$$Y = (A + \overline{B} + C) \cdot (A + \overline{B} + \overline{C}) \cdot (\overline{A} + \overline{B} + C)$$

$$Y(A, B, C) = \prod M(2, 3, 6)$$

To get Complement, Write only the left out Terms

$$\overline{Y}(A, B, C) = \prod M(0, 1, 4, 5, 7)$$

50

50

25

Positive Logic Vs Negative Logic

Positive Logic		Negative Logic	
Voltage Level	Logical Value	Voltage Level	Logical Value
LOW	0	LOW	1
HIGH	1	HIGH	0

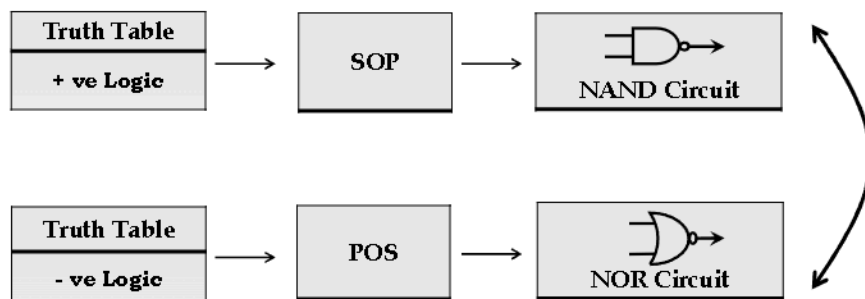
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

A	B	Y
1	1	1
1	0	1
0	1	1
0	0	0

51

51

SOP Vs POS



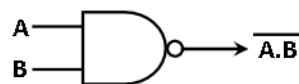
52

52

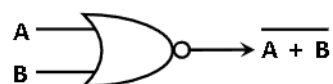
Universal Logic Elements

- ♦ NAND & NOR Gate - Universal Logic Elements
- ♦ Any Boolean Expression can be realized using these gates alone .

NAND Gate



NOR Gate

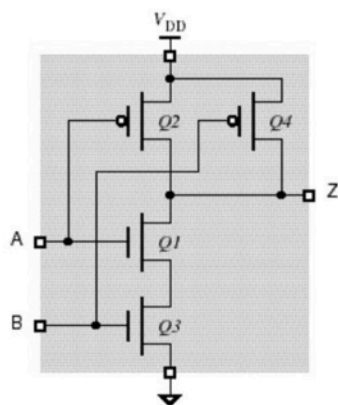


53

53

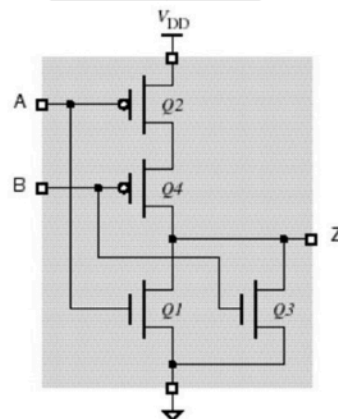
NAND Vs NOR

NAND



NAND - Faster and Smaller

NOR



54

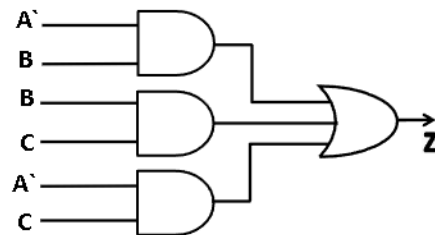
54

27

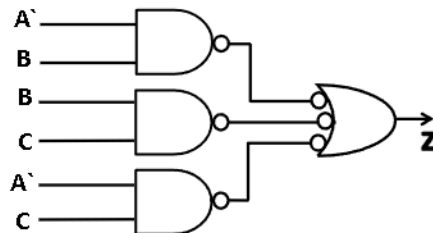
SOP - NAND Gates

Example : $Z = \overline{A}B + BC + \overline{A}C$

Step 1: Implement the Logic Circuit Normally.



Step 2: Replace the intermediate connecting wires with a pair of inverters connected in cascade.



55

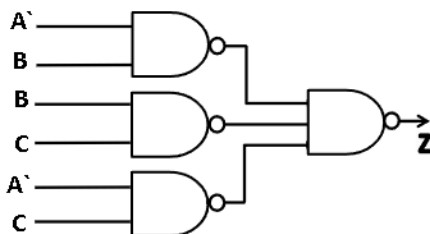
55

SOP - NAND Gates

Example : $Z = \overline{A}B + BC + \overline{A}C$

Notice the fact that a pair of inverters in cascade doesn't alter the functionality

Step 3: Replace bubbled OR gate with NAND Gate.



Now, the logic circuit consists of NAND gates only

56

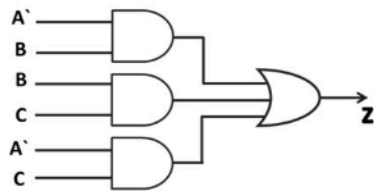
56

28

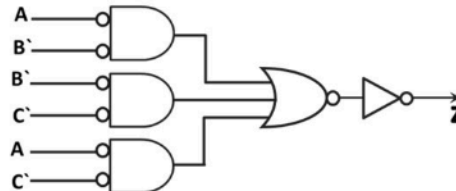
SOP - NOR Gates

Example : $Z = \overline{A}B + BC + \overline{A}C$

Step 1: Implement the Logic Circuit Normally.



Step 2: Replace the Input & Output wires with a pair of inverters connected in cascaded.



Notice the fact that inputs are also toggled to retain the functionality.

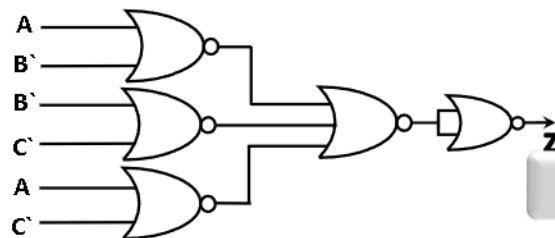
57

57

SOP - NOR Gates

Example : $Z = \overline{A}B + BC + \overline{A}C$

Step 3: replace the bubbled AND Gates at the inputs with NOR Gate & replace the Inverter with 2 i/p NOR Gate with both the i/p's shorted.



Now, the logic circuit consists of NOR gates only

58

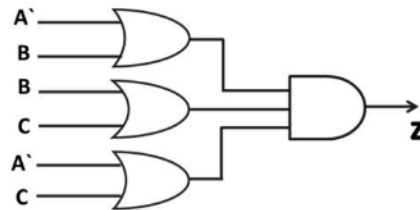
58

29

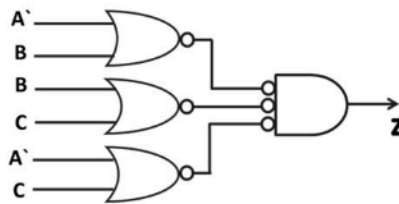
POS - NOR Gates

Example : $Z = (\bar{A} + B).(B + C).(\bar{A} + C)$

Step 1: Implement the Logic Circuit Normally.



Step 2: Replace the intermediate connecting wires with a pair of inverters connected in cascade.



59

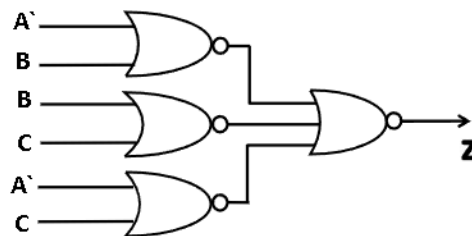
59

POS - NOR Gates

Example : $Z = (\bar{A} + B).(B + C).(\bar{A} + C)$

Notice the fact that a pair of inverters in cascade doesn't alter the functionality

Step 3: Replace bubbled AND gate with NOR Gate.



Now, the logic circuit consists of NOR gates only

60

60

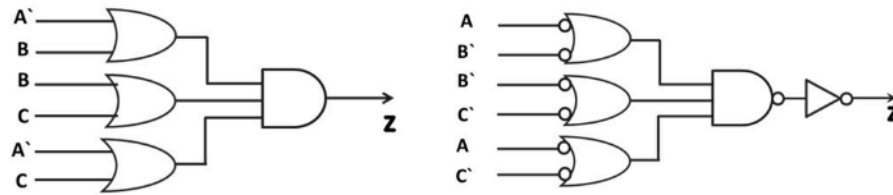
30

POS - NAND Gates

Example : $Z = (\bar{A} + B).(B + C).(\bar{A} + C)$

Step 1: Implement the Logic Circuit Normally.

Step 2: Replace the Input & Output wires with a pair of inverters connected in cascaded.



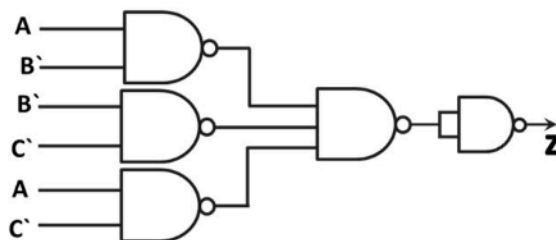
61

61

POS - NAND Gates

Example : $Z = (\bar{A} + B).(B + C).(\bar{A} + C)$

Step 3: replace the bubbled OR Gates at the inputs with NAND Gate & replace the Inverter with 2 i/p NAND Gate with both the i/p's shorted.



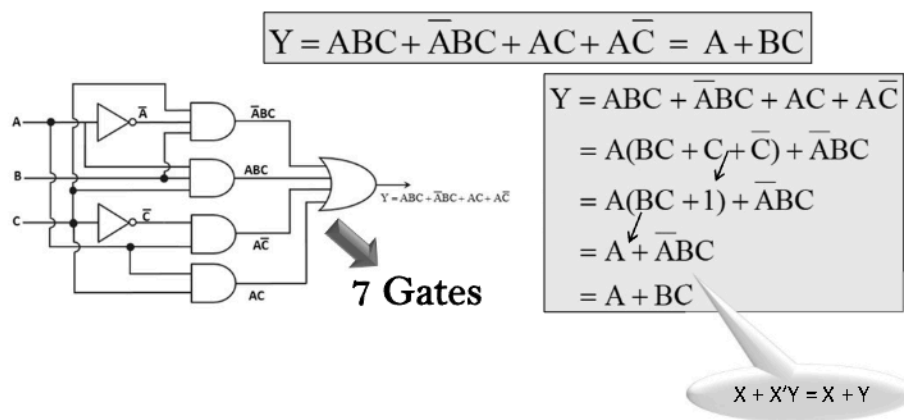
Now, the logic circuit consists of NAND gates only

62

62

31

Implement the function



63

63

Logic Minimization

Advantages of Logic Minimization

- ♦ Less no. of gates for implementation.
- ♦ Size of the circuit will be less.
- ♦ Less amount of power consumption.
- ♦ More speed of operation of the Logic Circuit.

A major step in digital design

64

64

32

Logic Minimization

Minimize the following expression

$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC$$

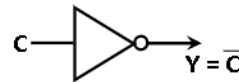
$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC$$

$$Y = (\overline{A}\overline{B} + \overline{A}B + A\overline{B} + AB)\overline{C}$$

$$Y = [\overline{A}(\overline{B} + B) + A(\overline{B} + B)]\overline{C}$$

$$Y = (\overline{A} + A)\overline{C}$$

$$Y = \overline{C}$$



65

65

Logic Minimization

Minimize the following expression

$$Y = B\overline{C}\overline{D} + A\overline{B}\overline{C} + A\overline{C}\overline{D} + A\overline{B}\overline{D} + \overline{A}\overline{B}\overline{D}$$

$$Y = B\overline{C}\overline{D} + A\overline{B}\overline{C} + A\overline{C}\overline{D} + A\overline{B}\overline{D} + \overline{A}\overline{B}\overline{D}$$

$$Y = B(\overline{C}\overline{D} + A\overline{C} + A\overline{D}) + A(\overline{B}\overline{C} + \overline{C}\overline{D} + \overline{B}\overline{D})$$

$$Y = B(A\overline{C} + A\overline{D}) + A(\overline{B}\overline{C} + \overline{B}\overline{D})$$

$$Y = A\overline{B}\overline{C} + \overline{A}\overline{B}\overline{D} + A\overline{B}\overline{C} + A\overline{B}\overline{D}$$

$$Y = A\overline{B}\overline{C} + \overline{A}\overline{B}\overline{D} + A\overline{B}\overline{D}$$

66

66

33

Karnaugh Map

CD AB	00 01 11 10			
	00	01	11	10
00	0000 0	0001 1	0011 3	0010 2
01	0100 4	0101 5	0111 7	0110 6
11	1100 12	1101 13	1111 15	1110 14
10	1000 8	1001 9	1011 11	1010 10

These cells marked with a specific numbers to represent the corresponding minterm or maxterm

Two adjacent cells are differed by only one digit because of the usage of gray code

67

67

Karnaugh Map

2 Variable K- Map

A \ B	0	1
	0	1
0	0	0
1	0	1

AB

3 Variable K- Map

A \ BC	00	01	11	10
	0	1	1	0
0	0	1	1	0
1	0	1	1	1

AB + C

4 Variable K- Map

CD AB	00	01	11	10
	0	1	1	0
00	0	1	0	0
01	0	1	0	0
11	1	1	1	1
10	0	1	0	0

AB + C'D

68

68

Karnaugh Map

$$F = \sum m(0, 1, 4, 6, 7)$$

		BC			
A		00	01	11	10
	0	1	1	0	0
	1	1	0	1	1

$$F = AB + A'B' + B'C'$$

Depending on the grouping,
multiple solutions are possible.

		BC			
A		00	01	11	10
	0	1	1	0	0
	1	1	0	1	1

$$F = AB + A'B' + AC'$$

69

69

Karnaugh Map

$$F = \sum m(0, 1, 7, 8, 9, 13, 14, 15)$$

		CD			
AB		00	01	11	10
	00	1	1	0	0
	01	0	0	1	0
	11	0	1	1	1
	10	1	1	0	0

$$F = B'C' + ABC + BCD + ABD$$

		CD			
AB		00	01	11	10
	00	1	1	0	0
	01	0	0	1	0
	11	0	1	1	1
	10	1	1	0	0

$$F = B'C' + ABC + BCD + AC'D$$

70

70

35

Karnaugh Map - POS

$$F = \prod M(1, 3, 4, 6, 11, 12)$$

		CD			
AB		00	01	11	10
00		1	0	0	1
01		0	1	1	0
11		0	1	1	1
10		1	1	0	1

$$F = (B' + C + D)(A + B + D')(B + C' + D')(A + B' + D)$$

71

71

Karnaugh Map

$$F = \prod M(1, 3, 4, 6, 11, 12) = \sum m(0, 2, 5, 7, 8, 9, 10, 13, 14, 15)$$

		CD			
AB		00	01	11	10
00		1	0	0	1
01		0	1	1	0
11		0	1	1	1
10		1	1	0	1

$$F = B'D' + BD + AC'D + ABC$$

		CD			
AB		00	01	11	10
00		1	0	0	1
01		0	1	1	0
11		0	1	1	1
10		1	1	0	1

$$F = B'D' + BD + AB'C' + ACD'$$

72

72

Don't Care Conditions

- ♦ An input-output conditions that never occurs during normal operation.
- ♦ It is indicated by 'X' in the truth table.
- ♦ It can be either 0 or 1, whichever produces simpler logic circuit.

73

73

Don't Care Conditions

$$F(A, B, C, D) = \sum m(2, 3, 4, 6, 11, 12) + d(10, 13, 14)$$

AB \ CD	CD			
	00	01	11	10
00	0	0	1	1
01	1	0	0	1
11	1	X	0	X
10	0	0	1	X

$$F = BD' + B'C$$

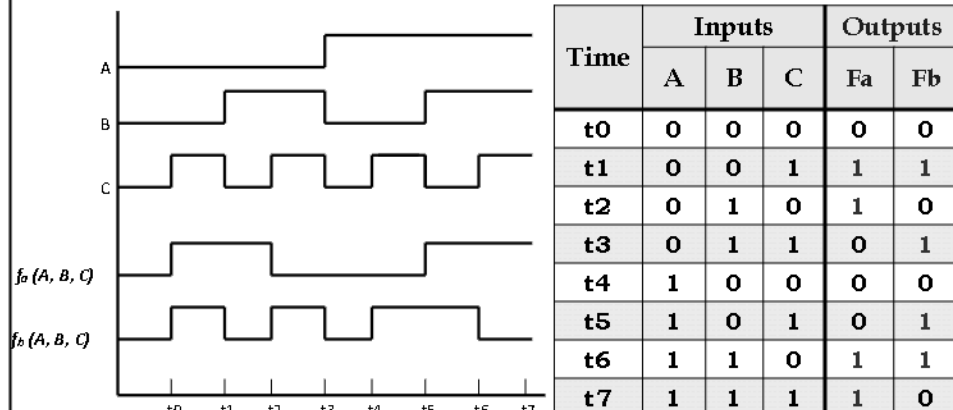
74

74

37

K Map Example

Design a circuit for the following output waveforms.



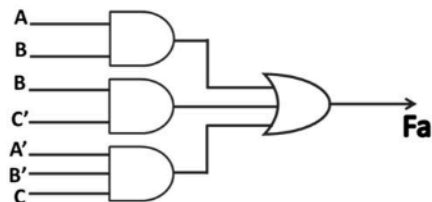
75

75

K Map Example

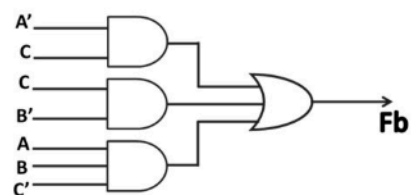
		BC			
A	0	00	01	11	10
	0	0	1	0	1
1	0	0	0	1	1

$$F_a = AB + BC' + A'B'C$$



		BC			
A	0	00	01	11	10
	0	0	1	1	0
1	0	0	1	0	1

$$F_b = A'C + B'C + ABC'$$



76

76

Combinational Circuits - I

77

77

Agenda

- ◆ Design of combinational circuits
- ◆ Delays in Combinational Circuits
- ◆ Adders & Subtractors

78

78

Digital Logic Circuits

Classification of Logic Circuits

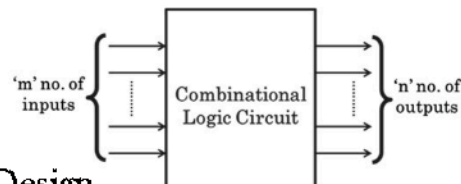
- ♦ **Combinational Logic Circuits**
 - ♦ Logic Circuits whose output is completely dependent on the combination of inputs.
- ♦ **Sequential Logic Circuits**
 - ♦ Logic Circuits whose output is dependent on the combination of inputs and past state of the circuit.
 - ♦ discussed in the later chapters.

79

79

Combinational Logic Circuits

Block Diagram of Combinational Circuits



Steps to Design

- ♦ Understand Problem description and Draw the Block Diagram with I/P & O/P lines.
- ♦ Define the truth table based on the functionality.
- ♦ Get the simplified expression.
- ♦ Implement the Logic circuit.

80

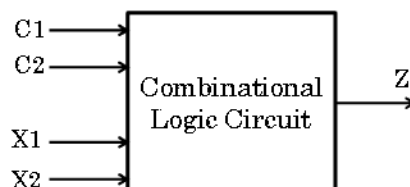
80

40

Combinational Logic Design

Example : Design a combinational circuit with the specifications given below.

A switching circuit has two control inputs (C1,C2),two data inputs (X1,X2) and one output (Z). The circuit performs one of the logic operations on the two data inputs. The function performed depends on the control inputs as shown below



C1	C2	Operation
0	0	OR
0	1	EXOR
1	0	AND
1	1	EQU

81

81

Truth Table

C1	C2	Operation
0	0	OR
0	1	EXOR
1	0	AND
1	1	EQU

With 4 input lines, 16 combinations of i/p's are possible.

C1	C2	X1	X2	Output (Z)	
0	0	0	0	0	OR
0	0	0	1	1	
0	0	1	0	1	
0	0	1	1	1	
0	1	0	0	0	EXOR
0	1	0	1	1	
0	1	1	0	1	
0	1	1	1	0	
1	0	0	0	0	AND
1	0	0	1	0	
1	0	1	0	0	
1	0	1	1	1	
1	1	0	0	1	EXNOR
1	1	0	1	0	
1	1	1	0	0	
1	1	1	1	1	

$$f(C1, C2, X1, X2) = \sum m(1, 2, 3, 5, 6, 11, 12, 15)$$

82

82

41

Minimization

$$f(C1, C2, X1, X2) = \sum m(1, 2, 3, 5, 6, 11, 12, 15)$$

		X1, X2			
		00	01	11	10
C1, C2	00	0	1	1	1
	01	0	1	0	1
	11	1	0	1	0
	10	0	0	1	0

Possible groupings:

$$Z = C1'X1'X2 + C1'X1X2' + C1X1X2 + C1'C2'X2 + C1C2X1'X2'$$

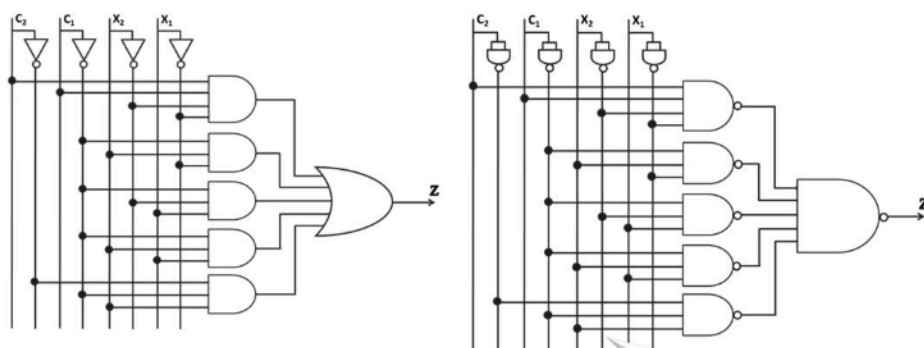
$$Z = C1'X1'X2 + C1'X1X2' + C1C2X1'X2' + C1X1X2 + C1'C2'X1$$

$$Z = C1'X1'X2 + C1'X1X2' + C1C2X1'X2' + C1X1X2 + C2'X1X2$$

83

83

Implementation

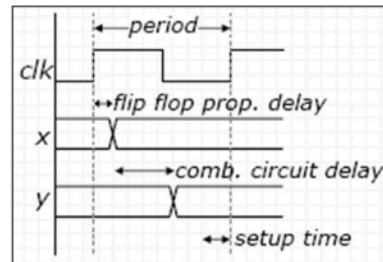
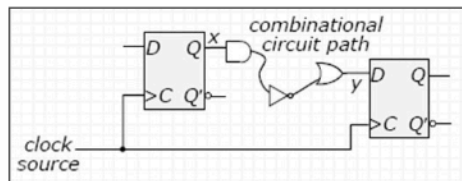


84

84

42

RTL Design



$$\text{Clock Period } T = T_{\text{hold}} + T_{\text{pd}} + T_{\text{setup}}$$

$$\text{Frequency} = 1 / T$$

85

85

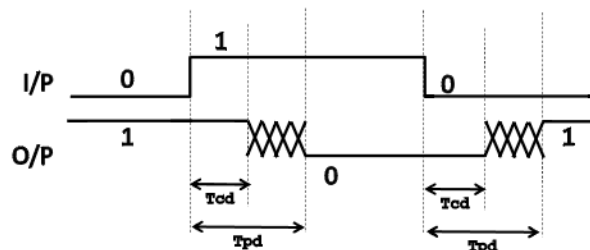
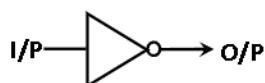
Delays in Combinational Circuits

Contamination Delay (T_{cd})

The amount of time needed for a change in a logic input to result in an initial change at an output.

Propagation Delay (T_{pd})

The amount of time needed for a change in a logic input to result in a permanent change at an output.



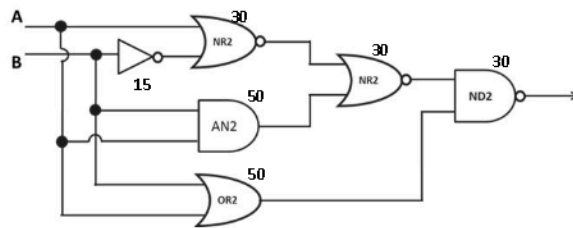
86

86

43

Delays in Combinational Circuits

Example : Using the following table of timing specifications for each component, what is the TPD value for the circuit shown below?



Gate	T_{cd}	T_{pd}
INV	3ps	15ps
ND2	5ps	30ps
AN2	12ps	50ps
NR2	5ps	30ps
OR2	12ps	50ps

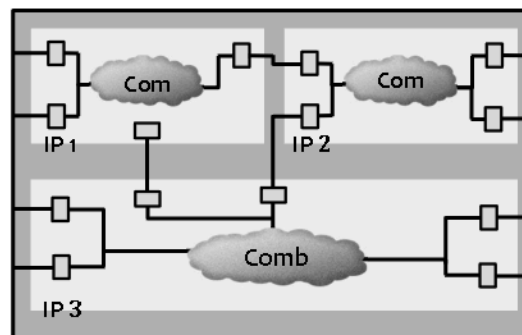
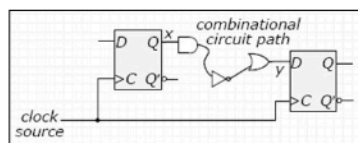
T_{pd}	90	105	110	80
	NR2, NR2, ND2	INV, NR2, NR2, ND2	AN2, NR2, ND2	OR2, ND2

110 ps is the maximum delay. So, the T_{pd} of this circuit is 110ps.

87

87

RTL Design



$$\text{Clock Period } T = T_{\text{hold}} + T_{pd} + T_{\text{setup}}$$

$$\text{Frequency} = 1 / T$$

88

88

44

Adders & Subtractors

89

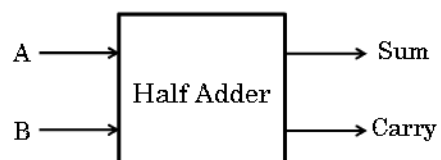
89

Adders – Half Adders

Specifications of Half Adder

Adds 2 bits - A,B & produces Sum & Carry as Outputs

Block Diagram, Truth Table, Boolean Expressions



A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$Sum = \bar{A}B + A\bar{B} = A \oplus B$$

$$Carry = AB$$

90

90

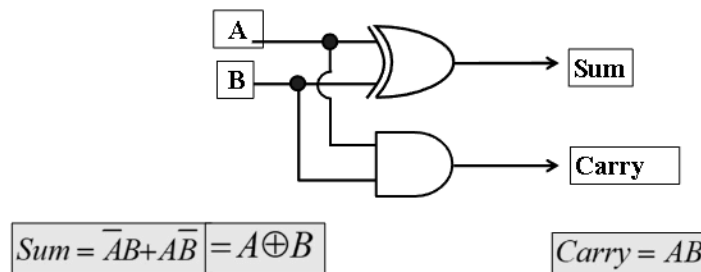
45

Adders – Half Adders

Specifications of Half Adder

Adds 2 bits - A,B & produces Sum & Carry as Outputs

Circuit Diagram



91

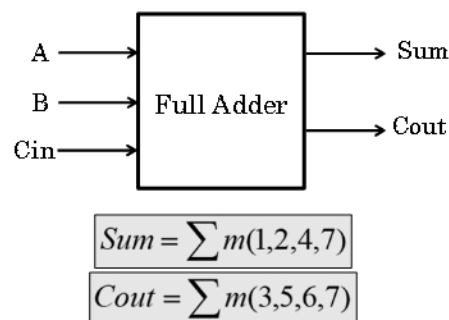
91

Adders – Full Adders

Specifications of Full Adder

Adds 3 bits - A,B & Carry-in and produces Sum & Carry-out as Outputs

Block Diagram, Truth Table, Boolean Expressions



A	B	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

92

92

Implementation of Full Adder

Boolean Expressions

$$Sum = \sum m(1,2,4,7)$$

A \ BC	BC			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$Sum = AB'C + A'B'C + ABC + A'BC'$$

$$Sum = A \oplus B \oplus C$$

$$Cout = \sum m(3,5,6,7)$$

A \ BC	BC			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$Cout = AB + BC + AC$$

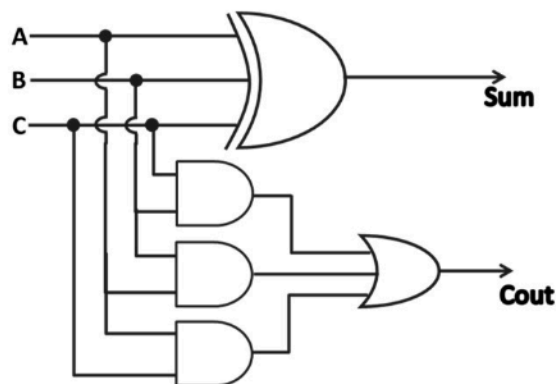
$$Cout = AB + BC + AC$$

93

93

Implementation of Full Adder

Logic Circuit



94

94

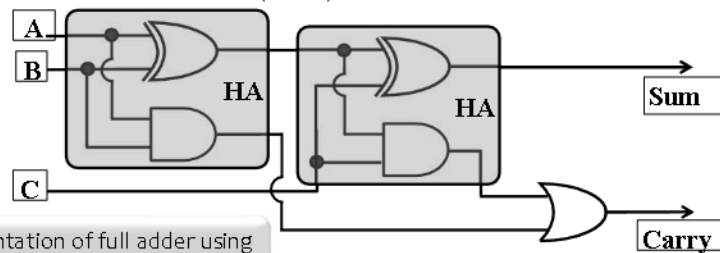
47

Another Version

$$Sum = A \oplus B \oplus C$$

$$Cout = AB + BC + AC$$

$$\begin{aligned} Cout &= A.B + B.C + A.C \\ &= AB + A'BC + ABC + AB'C + ABC \\ &= AB(1 + C + C) + C(A'B + AB') \\ &= AB + C(A \oplus B) \end{aligned}$$



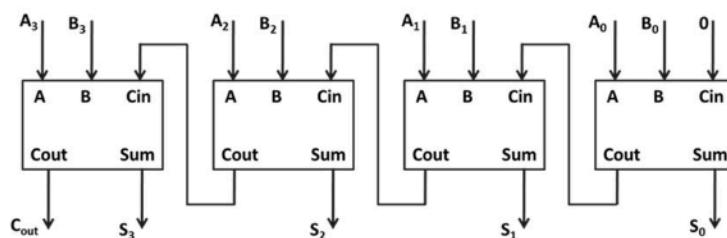
Implementation of full adder using
Two half adders & OR Gate.

95

95

4-bit Adder

		1	1	0				C_2	C_1	C_0		
10	→	1	0	1	0	A	→	A_3	A_2	A_1	A_0	
+ 7	→	+	0	1	1	1	+ B	→	B_3	B_2	B_1	B_0
17	←	1	0	0	0	1		C_{out}	S_3	S_2	S_1	S_0

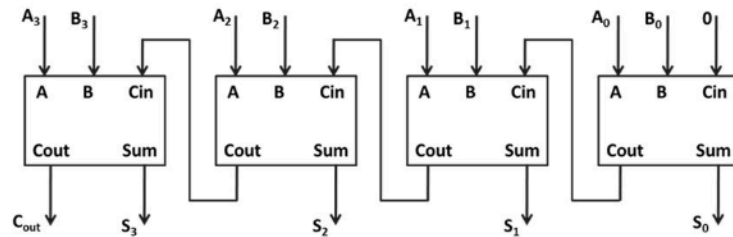


96

96

48

Delays in a 4-bit Adder

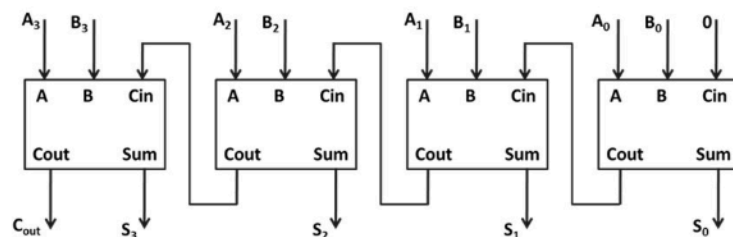


- Carry is propagated as a ripple from one stage to other stage. So, this adder is called as “ripple carry adder”
- In this adder, the addition process in each stage is waiting for the carry to be arrived from the previous stage of addition.
- So, the propagation delay of this adder is 4 times the Tpd of each adder. Hence, this adder is slow.

97

97

Delays in a 4-bit Adder



- To make the addition process much faster, we need to make the process of addition independent on the previous stage addition.
- Carry Look Ahead Adder is an adder in which, addition process in each stage is carried out without having a dependency on the previous stage.

98

98

4-bit Carry Look Ahead Adder

Truth Table of Full Adder

A	B	Cin	C-out	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$P = A \oplus B$$

$$G = AB$$

A + B generates carry, if and only if both A and B are 1

A + B propagates carry, if either one of A or B is 1

99

99

4-bit Carry Look Ahead Adder

$$P = A \oplus B$$

$$G = AB$$

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = A_i B_i + C_{i-1} (A_i \oplus B_i)$$

$$S_i = P_i \oplus C_{i-1}$$

$$C_i = G_i + C_{i-1} P_i$$

$$S_0 = P_0 \oplus C_{-1} \Rightarrow S_0 = P_0$$

$$C_0 = G_0 + C_{-1} P_0 \Rightarrow C_0 = G_0$$

$$S_1 = P_1 \oplus C_0 \Rightarrow S_1 = P_1 \oplus G_0$$

$$C_1 = G_1 + C_0 P_1 \Rightarrow C_1 = G_1 + G_0 P_1$$

$$S_2 = P_2 \oplus C_1$$

$$C_2 = G_2 + C_1 P_2$$

$$\Rightarrow S_2 = P_2 \oplus (G_1 + G_0 P_1)$$

$$\Rightarrow C_2 = G_2 + (G_1 + G_0 P_1) P_2$$

$$S_3 = P_3 \oplus C_2$$

$$C_3 = G_3 + C_2 P_3$$

$$\Rightarrow S_3 = P_3 \oplus (G_2 + (G_1 + G_0 P_1) P_2)$$

$$\Rightarrow C_3 = G_3 + (G_2 + (G_1 + G_0 P_1) P_2) P_3$$

100

100

50

4-bit Carry Look Ahead Adder

$$P = A \oplus B \quad G = AB$$

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$S_i = P_i \oplus C_{i-1}$$

$$\Rightarrow S_0 = P_0$$

$$\Rightarrow S_1 = P_1 \oplus G_0$$

$$\Rightarrow S_2 = P_2 \oplus (G_1 + G_0 P_1)$$

$$\Rightarrow S_3 = P_3 \oplus (G_2 + (G_1 + G_0 P_1) P_2)$$

All sum & carry expressions are independent of any carry from the previous stage.

$$C_i = A_i B_i + C_{i-1} (A_i \oplus B_i)$$

$$C_i = G_i + C_{i-1} P_i$$

$$\Rightarrow C_0 = G_0$$

$$\Rightarrow C_1 = G_1 + G_0 P_1$$

$$\Rightarrow C_2 = G_2 + (G_1 + G_0 P_1) P_2$$

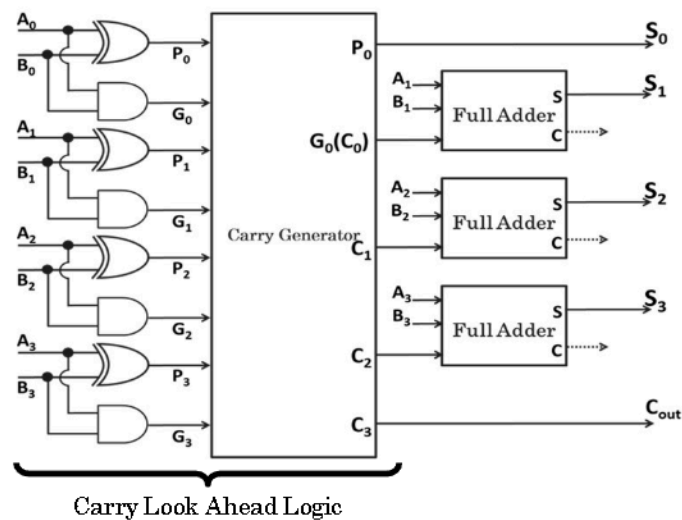
$$\Rightarrow C_3 = G_3 + (G_2 + (G_1 + G_0 P_1) P_2) P_3$$

So, these expressions can be used to realize the hardware of the adder to improve the speed.

101

101

4-bit Carry Look Ahead Adder



102

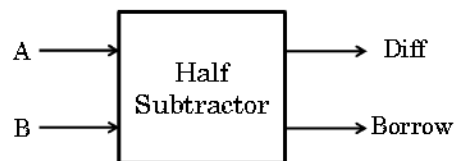
102

Subtractors – Half Subtractor

Specifications of Half Subtractor

Subtracts B from A & produces Difference & Borrow as Outputs

Block Diagram, Truth Table, Boolean Expressions



A	B	Borrow	Diff
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

$$Diff = \bar{A}B + A\bar{B} = A \oplus B$$

$$Borrow = \bar{A}B$$

103

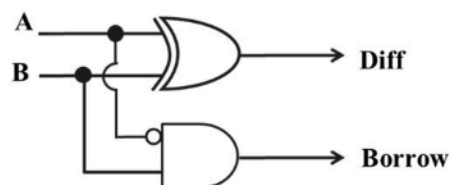
103

Subtractors – Half Subtractor

Specifications of Half Subtractor

Subtracts B from A & produces Difference & Borrow as Outputs

Circuit Diagram



$$Diff = \bar{A}B + A\bar{B} = A \oplus B$$

$$Borrow = \bar{A}B$$

104

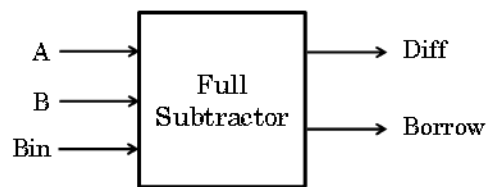
104

Adders – Full Subtractor

Specifications of Full Subtractor

Subtracts B and Borrow-in from A

Block Diagram, Truth Table, Boolean Expressions



$$Diff = \sum m(1,2,4,7)$$

$$Borrow = \sum m(1,2,3,7)$$

A	B	B-in (C)	Borrow	Diff
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

105

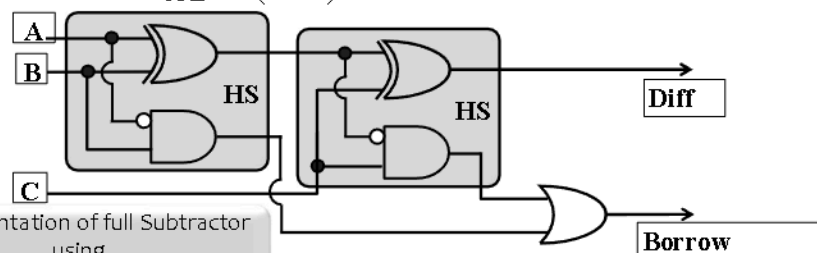
105

Another Version

$$Diff = A \oplus B \oplus C$$

$$Borrow = \bar{A}B + BC + \bar{A}C$$

$$\begin{aligned}
 Borrow &= A'B + A'C + B.C \\
 &= A'B + A'BC + A'B'C + A'BC + ABC \\
 &= A'B(1 + C + C) + C(A'B' + AB) \\
 &= A'B + C(A \wedge B)'
 \end{aligned}$$



Implementation of full Subtractor using Two half subtractors & OR Gate.

106

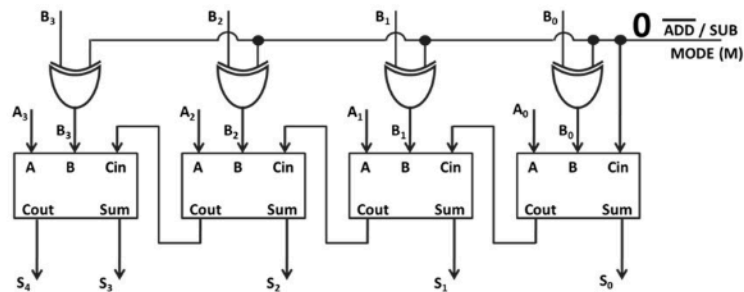
106

53

Adder Subtractor Circuit

$$A + B \Rightarrow A + B \Rightarrow A + B + 0$$

$$A - B \Rightarrow A + 2's \text{ Comp}(B) \Rightarrow A + 1's \text{ Comp}(B) + 1$$



$$\text{Output} = A + B + 0 = \text{Sum}$$

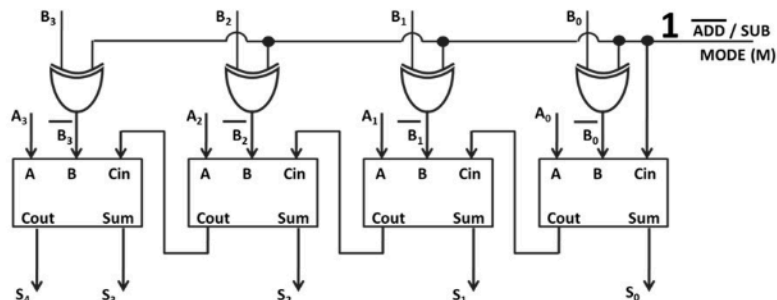
107

107

Adder Subtractor Circuit

$$A + B \Rightarrow A + B \Rightarrow A + B + 0$$

$$A - B \Rightarrow A + 2's \text{ Comp}(B) \Rightarrow A + 1's \text{ Comp}(B) + 1$$



$$\text{Output} = A + \overline{B} + 1 = \text{Diff}$$

108

108

Combinational Circuits - II

109

109

Agenda

- ◆ Encoder & Decoder
- ◆ Mux & DeMux
- ◆ Tristate Buffer

110

110

Encoders

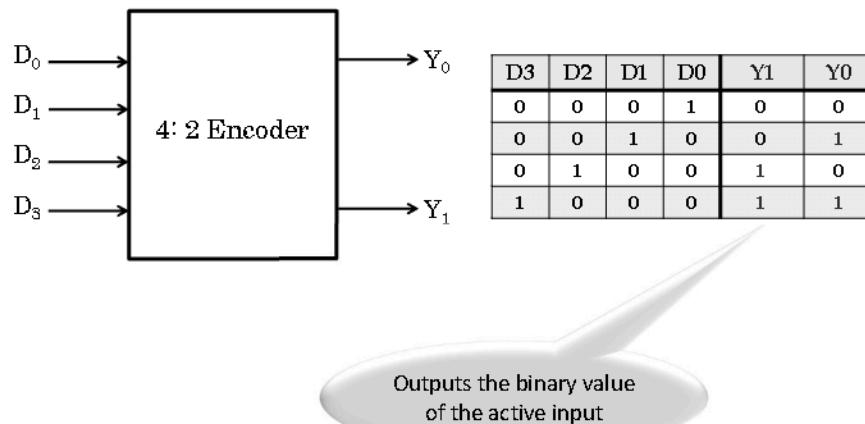
Encoders

- ◆ Converts Human understandable into machine understandable codes
- ◆ Assigns a binary code to an active input line.
- ◆ One hot to binary converter,
- ◆ Produces n no. of outputs when there is 2^n no. of inputs,
- ◆ At most only one of the inputs will ever be high, the binary code of this 'hot' line is produced

111

111

4:2 Encoder



112

112

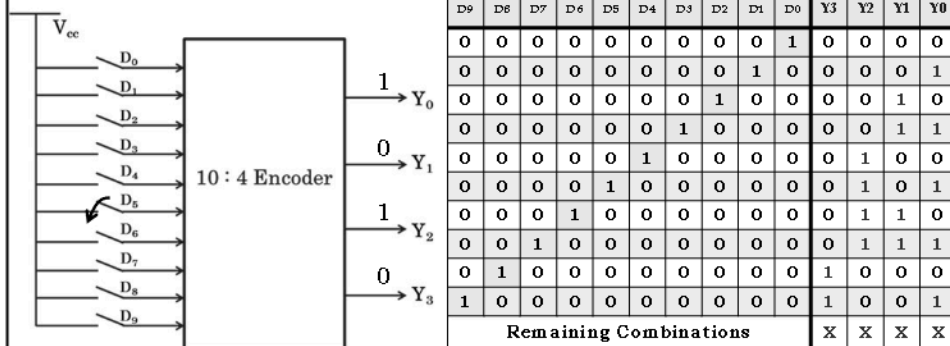
Decimal to Binary Encoder

Specifications

- ♦ Take 10 keys as Decimal inputs & produce 4 bit Binary Output based on the input key

What if more than one input is active?
What if no inputs are active?

Block Diagram, Truth Table



113

113

Priority Encoder

Priority Encoders

- ♦ Assign priorities to the inputs when more than one input are asserted simultaneously,
- ♦ Highest priority input is taken into account to produce the binary output.
- ♦ At least any one of the input line should be active, to make the encoder valid. A 'valid' indicator, is included to indicate the same.

114

114

8:3 Priority Encoder

Priority to higher order inputs.

D7	D6	D5	D4	D3	D2	D1	D0	Y2	Y1	Y0	Valid
0	0	0	0	0	0	0	0	X	X	X	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

115

115

ASCII Encoder

♦ ASCII stands for American Standard Code for Information Interchange

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	EF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	--	#	\$	%	&	.	{	}	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	:	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	/	]	^	_
6	.	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

♦ The code uses 7 bits to encode 128 unique characters

116

116

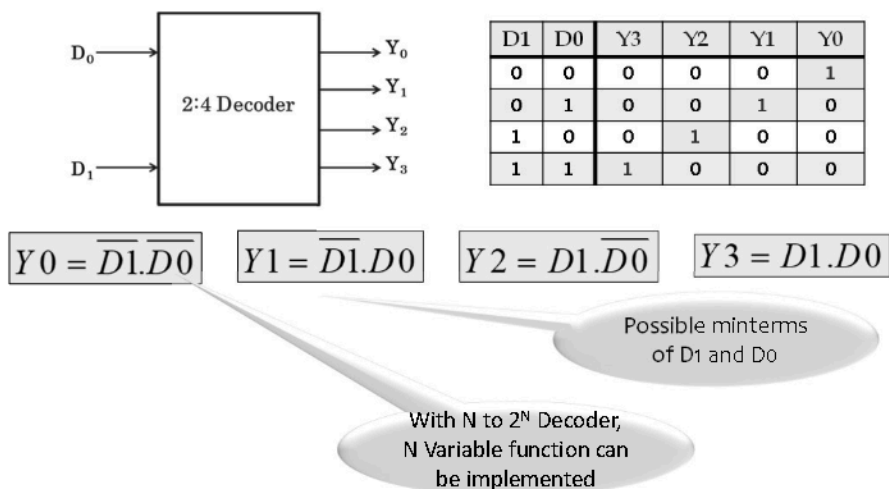
Decoders

- ◆ Covert Machine understandable into Human understandable codes
- ◆ Convert binary information from n input signals to 2^n unique output signals
- ◆ used in a wide variety of applications, including data demultiplexing, seven segment displays, and memory address decoding.

117

117

2:4 Decoder

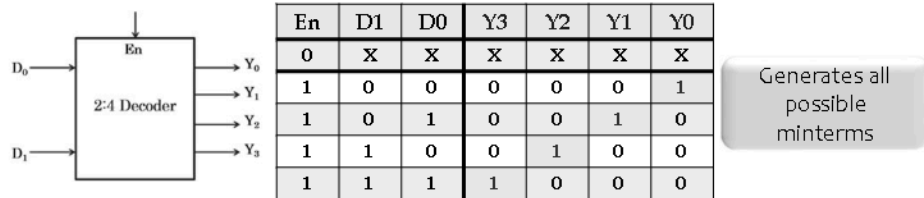


118

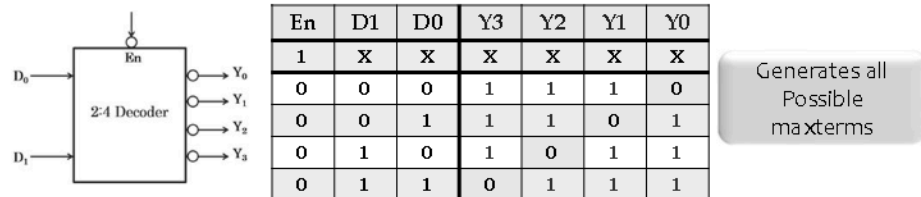
118

Decoders with Active High & Low O/P with Enable inputs.

Decoder with Active HIGH Output & active HIGH Enable



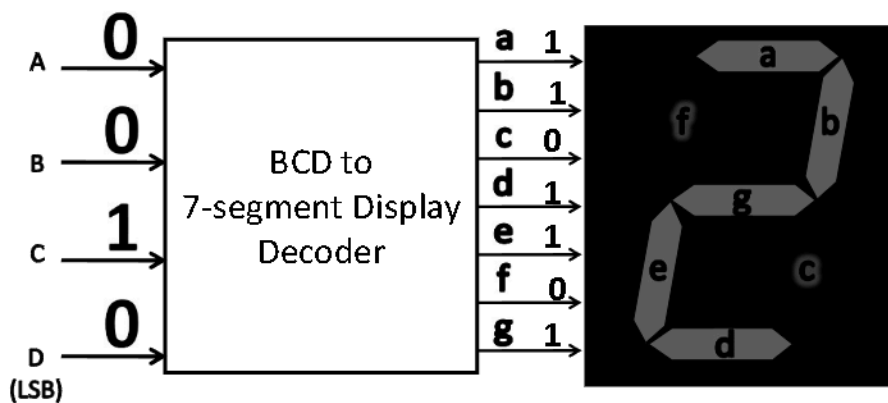
Decoder with Active LOW Output & Active LOW Enable



119

119

BCD-to-Seven Segment Decoder



120

120

Circuit Design using Decoders

- ♦ Any combinational circuit with n inputs and m outputs can be implemented with an n -to- 2^n decoder with m OR gates because decoder with active HIGH output produces all possible minterms.
- ♦ Similarly, Any combinational circuit with n inputs and m outputs can be implemented with an n -to- 2^n decoder with m AND gates because decoder with active LOW output produces all possible maxterms.

121

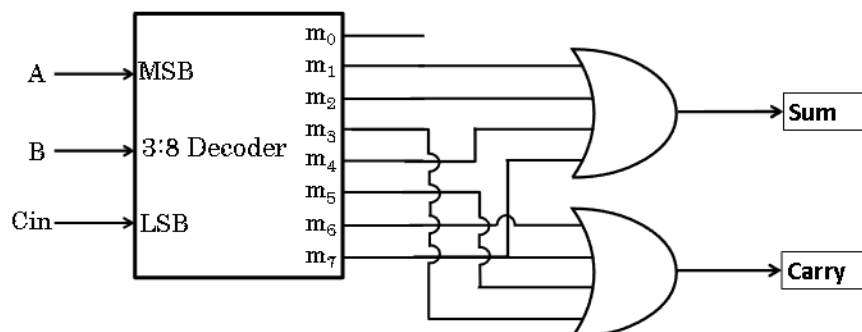
121

Fulladder using Decoders

Example : Realize a full adder using a 3 to 8 decoder using Active HIGH output decoder.

$$Sum = \sum m(1,2,4,7)$$

$$Cout = \sum m(3,5,6,7)$$



122

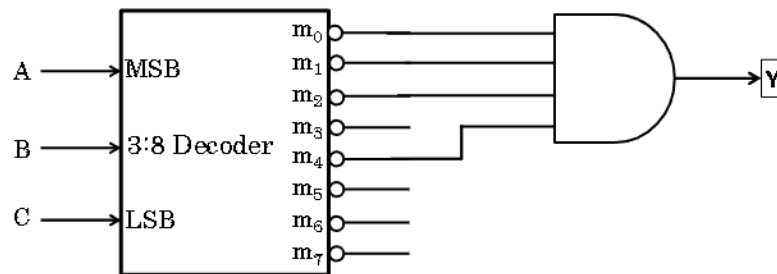
122

Boolean Functions using Decoders

Example : Implement the following function using an active LOW output decoder.

$$Y = \prod M(0,1,2,4)$$

For implementing any function using active low output decoder, the function should be in canonical POS form



123

123

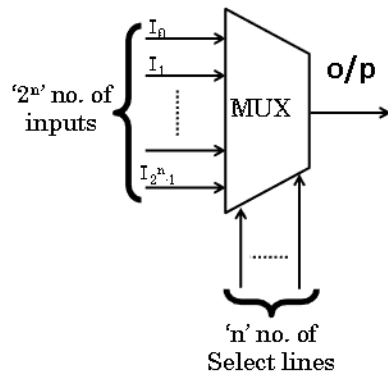
MUX & DeMUX

124

124

Multiplexer (MUX)

Logic Symbol & Functionality

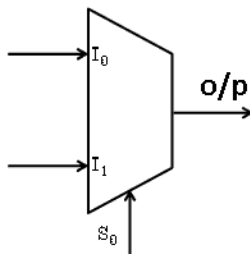


- ♦ Single output line
- ♦ One of the data input is routed to the output based on the select signals.

125

125

2:1 MUX



Functional Table

S_0	O/P
0	I_0
1	I_1

$$Y = S_0' I_0 + S_0 I_1$$

Truth Table

S_0	I_1	I_0	O/P
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

O/P same as I_0

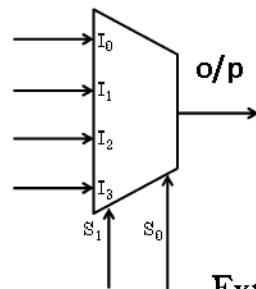
O/P same as I_1

$$\text{output } (Y) = \sum m(1, 3, 6, 7)$$

125

126

4:1 MUX



Functional Table

S_1	S_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Expression for output

$$Y = \overline{S_1} \overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

127

127

MUX – Universal Logic

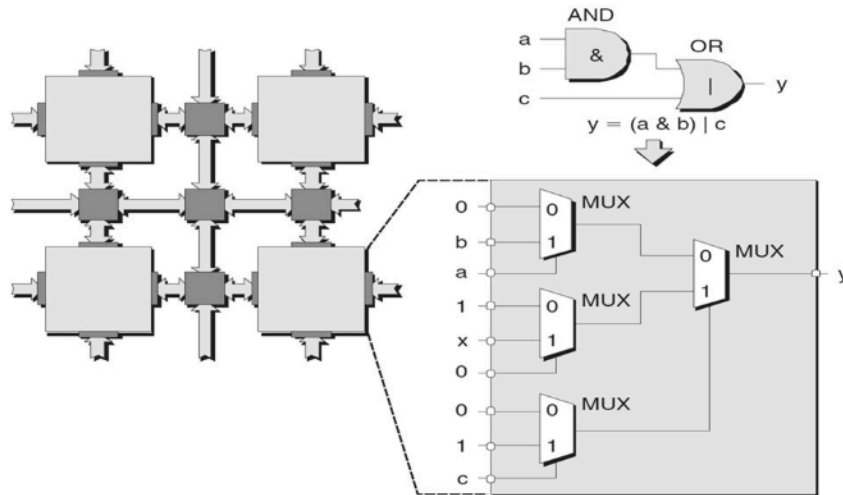
- ♦ Similar to NAND & NOR, MUX can also be used to realize any Boolean Function.
- ♦ MUX – Universal Logic Block

- ♦ Functions are in Canonical Form - Use truth tables
- ♦ Not in Canonical Form – Use Shannon's Expression.

128

128

MUX – Universal Logic



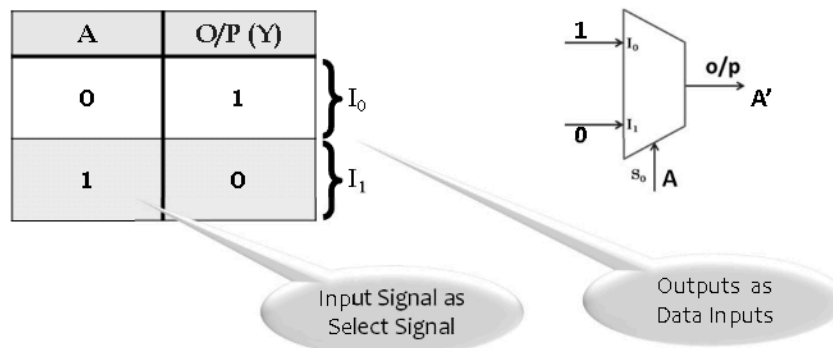
129

129

Implementing Logic Gates using MUX

MUX as NOT gate

- ♦ If input is A, then output should be complement of A.



130

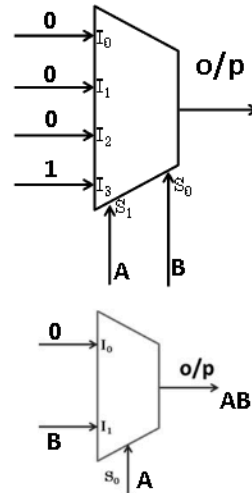
130

Implementing Logic Gates using MUX

MUX as AND gate

A	B	O/P (Y)
0	0	0
0	1	0
1	0	0
1	1	1

$I_0 = 0$
 $I_1 = B$

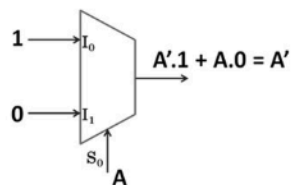


131

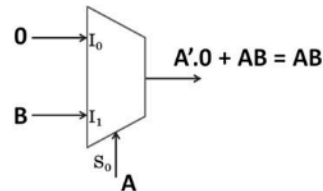
131

Implementing Logic Gates using MUX

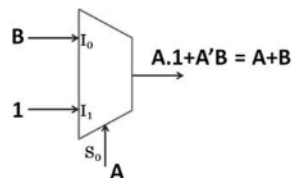
NOT gate



AND gate



OR gate

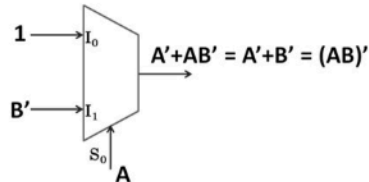


132

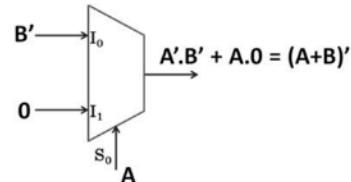
132

Implementing Logic Gates using MUX

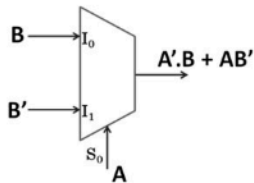
NAND gate



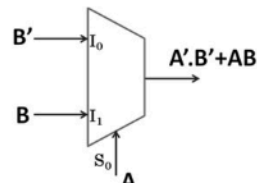
NOR gate



EXOR gate



EXNOR gate



133

133

Circuit Design Using MUX

Implement the following function using 16:1 MUX

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 6, 7, 9, 13, 14)$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

- Since, the no. of literals in the expression & the no. of select lines is 4, all the input literals can be used as select lines.

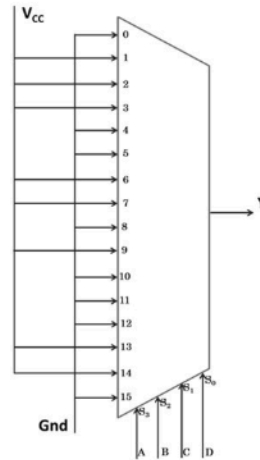
134

134

Function Implementation Using MUX

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 6, 7, 9, 13, 14)$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



135

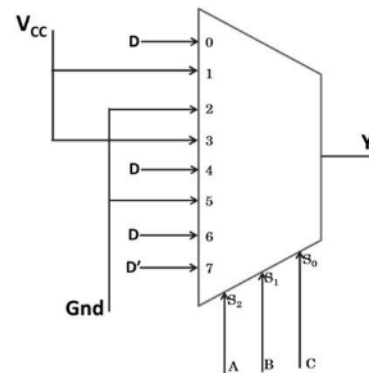
135

Implement using 8:1 MUX

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 6, 7, 9, 13, 14)$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

$I_0 = D$
 $I_1 = 1$
 $I_2 = 0$
 $I_3 = 1$
 $I_4 = D$
 $I_5 = 0$
 $I_6 = D$
 $I_7 = D'$



136

136

Implement Using 4: 1 MUX

$$Y = f(A, B, C, D) = \sum m(1, 2, 3, 6, 7, 9, 13, 14)$$

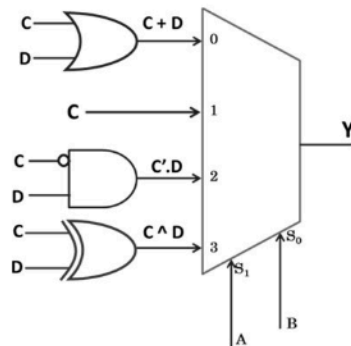
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

$$I_0 = C + D$$

$$I_1 = C$$

$$I_2 = C'D$$

$$I_3 = C \wedge D$$



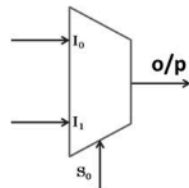
137

137

MUX - Shannon's Expression

2: 1 MUX

Expression for the output

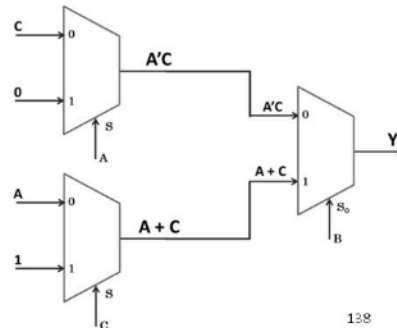


$$Y = \overline{S_0} I_0 + S_1 I_1$$

Known as Shannon's Expression for 2:1 MUX

$$Y = BC + AB \overline{C} + \overline{A} BC$$

1. Bring this expression into the form of Shannon's Expression.
2. Then draw using 2:1 MUX
3. You can learn this in detail during Advance Digital course



138

138

Hierarchical Design

- ♦ Several issues arise when designing large multiplexers(as 2-level circuits).
 - Number of logic gates gets prohibitively large
 - Number of inputs to each logic gate (i.e. fan-in) gets prohibitively large
- ♦ Instead, design hierarchically
 - Use smaller elements as building blocks
 - Interconnect building blocks in a multi-tier structure

139

139

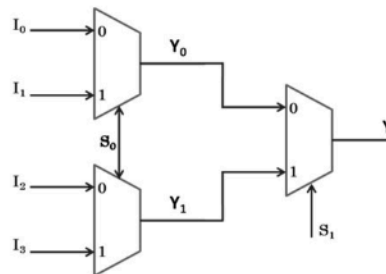
4:1 MUX using 2:1 MUX

Truth Table

S_1	S_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$\left. \begin{matrix} I_0 \\ I_1 \end{matrix} \right\} Y_0$
 $\left. \begin{matrix} I_2 \\ I_3 \end{matrix} \right\} Y_1$

S_1	O/P
0	Y_0
1	Y_1



1. I_0 or I_1 can be selected by using a 2:1 MUX using S_0 as the select line signal.
2. I_2 or I_3 can be selected by using another 2:1 MUX using S_0 as the select line signal.
3. Y_0 or Y_1 can be selected by using another 2:1 MUX using S_1 as the select line signal

140

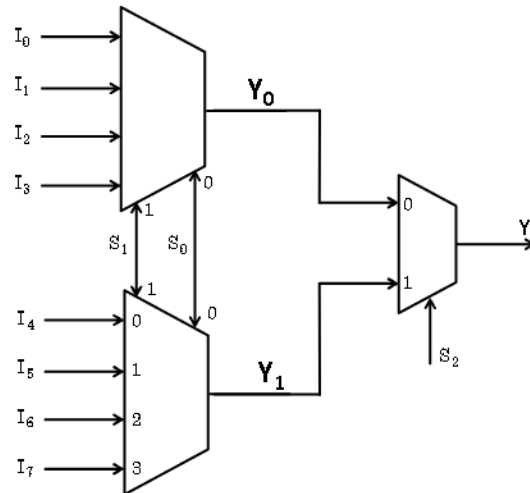
140

70

8:1 MUX using 4:1 & 2:1 MUX

Truth Table

S_2	S_1	S_0	O/P
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

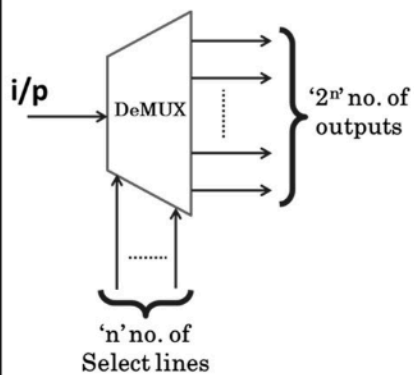


141

141

Demultiplexer (DeMUX)

Logic Symbol & Functionality

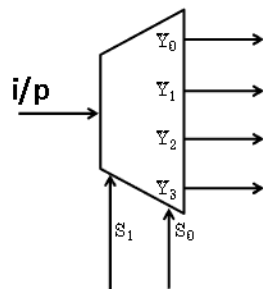


- ♦ It's operation is just opposite to that of a Multiplexer.
- ♦ Depending on the data on the select lines, the input will be redirected to any one specific output line.

142

142

1:4 DeMUX



Expression for output

$$Y_0 = \overline{S_1} \overline{S_0} I$$

$$Y_1 = \overline{S_1} S_0 I$$

$$Y_2 = S_1 \overline{S_0} I$$

$$Y_3 = S_1 S_0 I$$

143

143

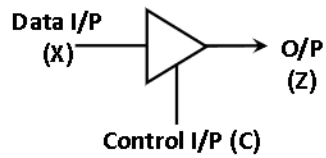
Tristate Buffer

144

144

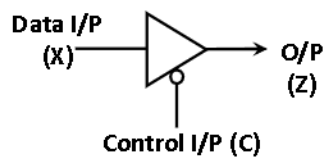
Tri State Buffers

Tristate Buffer with Active HIGH Control



C	X	Carry
0	0	Z
0	1	Z
1	0	0
1	1	1

Tristate Buffer with Active LOW Control

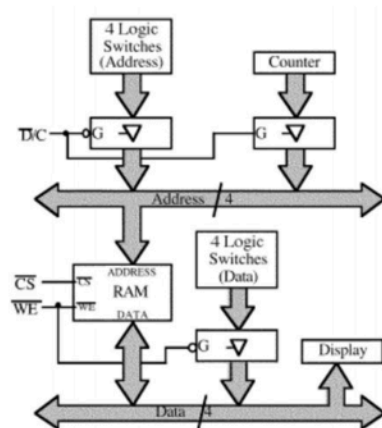


C	X	Carry
0	0	0
0	1	1
1	0	Z
1	1	Z

145

145

Memory Circuit



$\overline{D/C}$	Mode
0	Switches
1	Counter

$\overline{WE}$	Mode
0	Write
1	Read

146

146

73

Sequential Logic Circuits - I

147

147

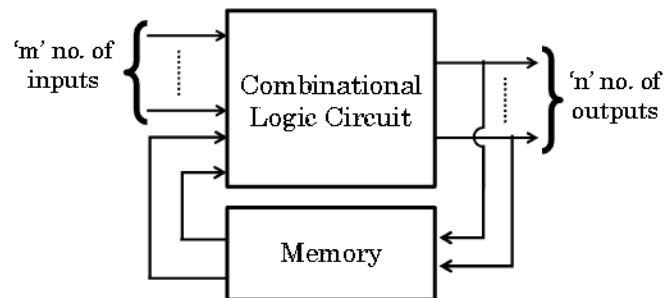
Agenda

- ◆ Sequential Logic circuits
- ◆ Latches
- ◆ Flipflops
- ◆ Delays in Sequential Circuits

148

148

Sequential Logic Circuits

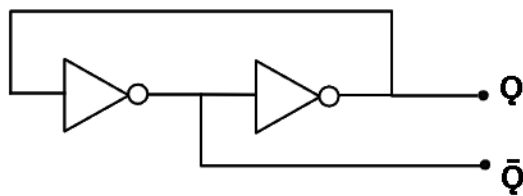


- ♦ The output depends on the input & past state of the circuit.
- ♦ Memory Units

149

149

Memory units



Q_n	Q_{n+1}
1	1
0	0

The above circuit is capable of restoring the data which is available at instant of power ON condition.

This indicates that cascaded combination of 2 inverters with a feedback connection is capable of storing 1 bit information.

150

150

75

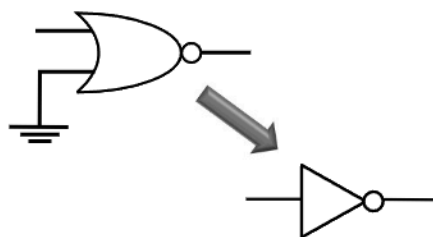
Logic Circuits as memory units

- ♦ The cascade connection of **2 inverters with a feedback** retains the past information
- ♦ And hence it can be **considered as a 1-bit memory element** or 1-bit memory cell.
- ♦ But, there is no input port and hence we don't have the control over the data that has to be stored.
- ♦ Let's try to **modify this circuit and provide i/p ports** by which we can customize the data that is going to be stored in the memory cell.

151

151

1-bit memory cell



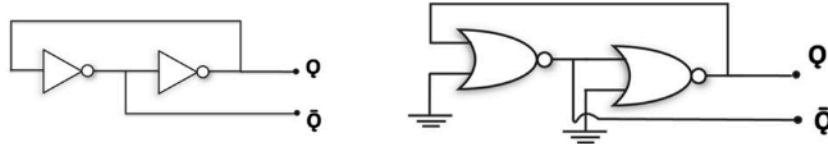
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- ♦ If $A = 0$, then $Y = B'$.
- ♦ This states that if one of the input to NOR gate is grounded, then that NOR gate will act as a NOT gate.

152

152

1-bit memory cell

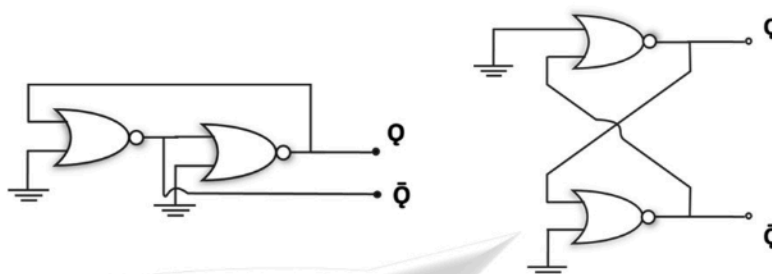


- ♦ In this memory cell, these two inverters can be replaced with a pair of NOR gates with one input connected to ground.
- ♦ In this circuit, the feedback is in such a way that, output of first inverter is input to the second inverter & output of second inverter is the input to the first inverter.

153

153

1-bit memory cell

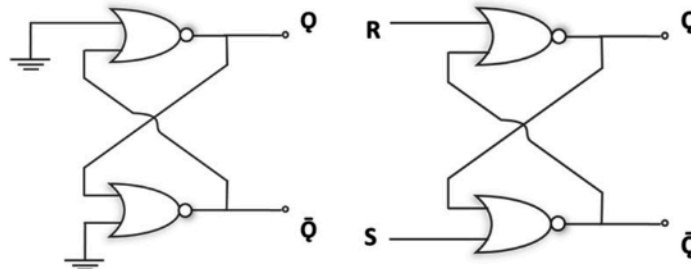


But, even with modification, there is no input port and hence we don't have the control over the data to be stored.

154

154

1-bit memory cell

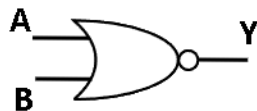


- ♦ This is how, RS Latch has been evolved.
- ♦ Latch is nothing but a one bit memory cell which is capable of retaining the past state output.
- ♦ Since, latch is evolved from a cascaded combination of a pair of inverters, the two outputs of a latch should be always complement to each other.

155

155

Behaviour of a NOR gate



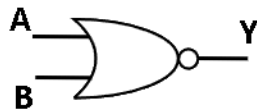
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- ♦ This states that if one of the input to NOR gate is 0, then the output of the NOR gate will be complement to the remaining input.
- ♦ This indicates that, if one of the input is 0, then we cannot predict the output of the NOR gate without having the knowledge of the remaining input.

156

156

Behaviour of a NOR gate



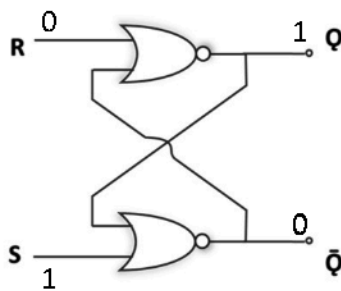
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- ♦ This states that if one of the input to NOR gate is 1, then the output of the NOR gate will be always 0 irrespective of the remaining input.
- ♦ This indicates that, if one of the input is 1, then we can easily predict the output of the NOR gate without having the knowledge of the remaining input.

157

157

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set

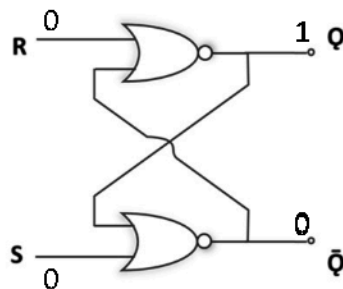
Recollect the fact that, if one of the input to NOR is 1, then output will be '0'.

The outputs for this input combination is 1 & 0.

158

158

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change

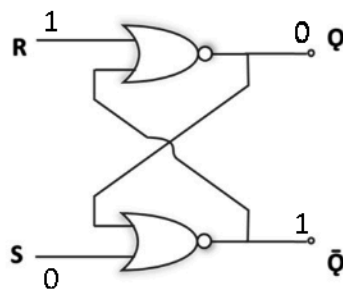
In this case also, the outputs for this input combination is 1 & 0.

Even though, there is some change input, the output is same as the output in the past state

159

159

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset

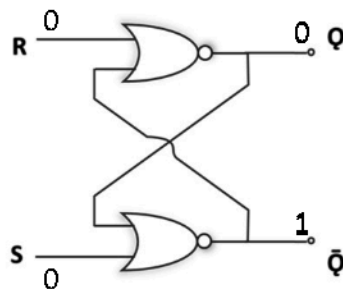
Recollect the fact that, if one of the input to NOR is 1, then output will be '0'.

The outputs for this input combination is 0 & 1.

160

160

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset
0	0	0	1	No Change

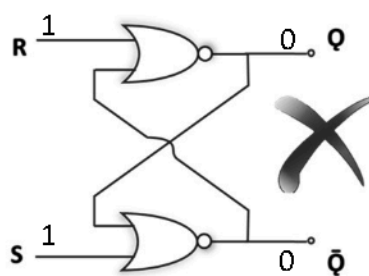
In this case also, the outputs for this input combination is 0 & 1.

Even though, there is some change in the input, the output is same as the output in the past state

161

161

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset
0	0	0	1	No Change
1	1	0	0	Invalid

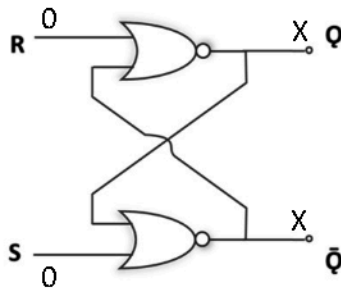
Observe what happens if invalid combination of i/p is followed by no change combination of input.

Both the outputs of a latch should be complement to each other. But, in this case, both the outputs are same.

162

162

RS Latch



R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset
0	0	0	1	No Change
1	1	0	0	Invalid
0	0	X	X	RACE

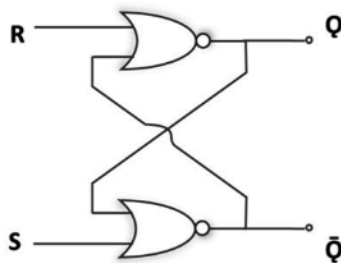
Q	1	0	1	0	1
Q'	1	0	1	0	1

1. At the o/p terminals, both 1 & 0 will be racing with each other even though there is no change in the i/p .
2. If invalid combination of i/p is followed by no change combination of i/p, race around condition occurs.

163

163

RS Latch



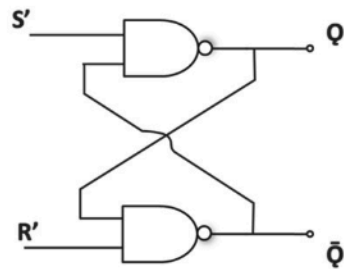
R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset
0	0	0	1	No Change
1	1	0	0	Invalid
0	0	-	-	RACE

1. Since, the Set input is HIGH, output is SET.
2. Since, the Reset input is HIGH, output is RESET.
3. That's why, RS Latch with NOR Gates is said to be Active HIGH Latch.

164

164

RS Latch – NAND Gates



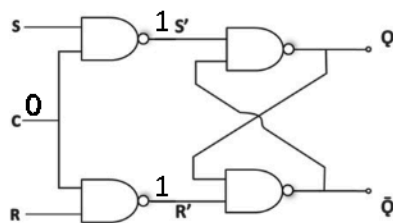
S'	R'	Q	Q'	State
0	1	1	0	Set
1	1	1	0	No change
1	0	0	1	Reset
1	1	0	1	No change
0	0	1	1	Invalid
1	1	-	-	RACE

- ♦ The only difference is RS Latch with NOR gates is active HIGH Latch where as RS Latch with NAND gates is active LOW Latch.
- ♦ In this case, (0 & 0) input combination is an invalid combination of input and (1 & 1) input combination doesn't bring any change in the output.

165

165

Gated RS Latch

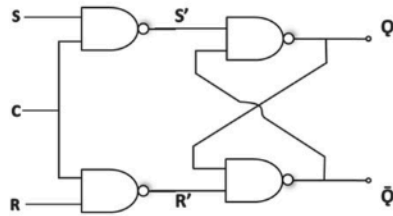


C	S	R	S'	R'	Q	Q'	State
0	X	X	1	1			No Change
1	0	0	1	1			No Change
1	0	1	1	0	0	1	Reset
1	1	0	0	1	1	0	Set
1	1	1	0	0	-	-	Invalid

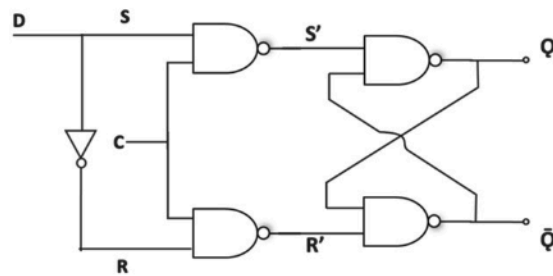
166

166

Gated D Latch



C	S	R	S'	R'	Q	Q'	State
0	X	X	1	1			No Change
1	0	0	1	1			No Change
1	0	1	1	0	0	1	Reset
1	1	0	0	1	1	0	Set
1	1	1	0	0	-	-	Invalid



C	D	Q	Q'	State
0	X			No Change
1	0	0	1	Reset
1	1	1	0	Set

167

167

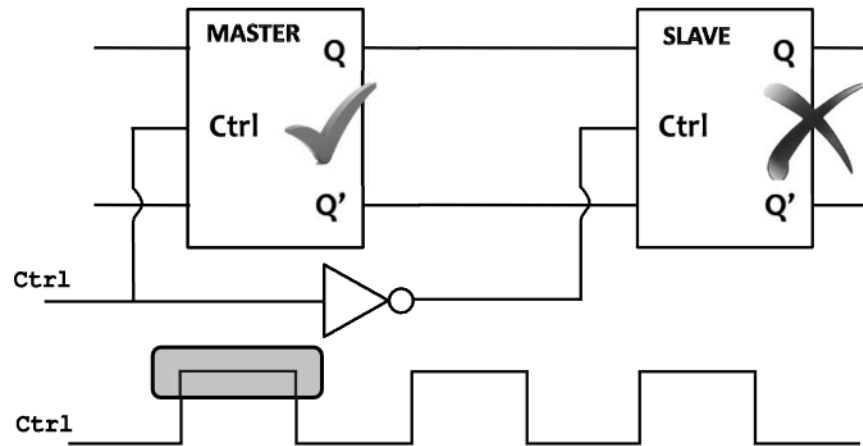
Gated D Latch

- ♦ In Controlled RS Latch, RACE Condition will occur if both inputs are 1.
- ♦ In controlled RS Latch, if both the inputs are complement to each other, the output follows the S input.
- ♦ So, to avoid RACE conditions, both the inputs must be complement to each other.
- ♦ This is how D Latch is constructed.
- ♦ Output Q follows the input D, said to be transparent Latch.

168

168

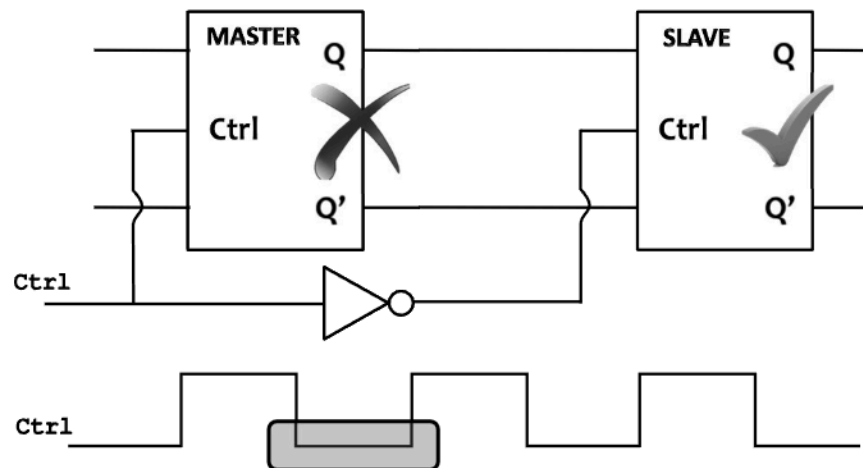
Master Slave Configuration



169

169

Master Slave Configuration



170

170

Master Slave Configuration

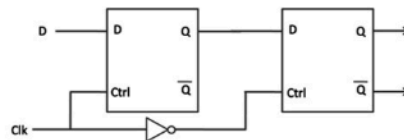
- ♦ The output at the final stage will be depending on the input available at the instant of negative edge.
- ♦ Master Slave configuration of two latches is treated as a flip flop.

171

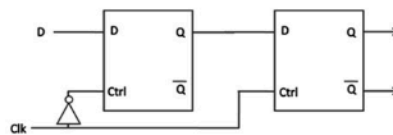
171

Construction of Flipflops

- ♦ Negative Edge Sensitive flipflop is constructed by using HIGH level latch as master and LOW level latch as slave.



- ♦ Positive Edge Sensitive flipflop is constructed by using LOW level latch as master and HIGH level latch as slave.



...

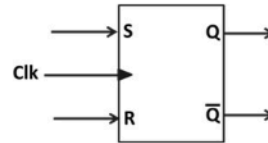
172

172

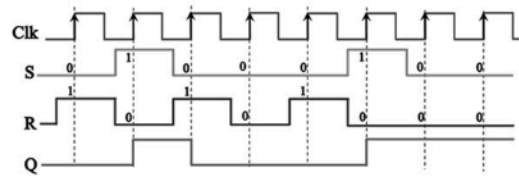
SR Flip-flop

Truth Table

Clk	S	R	Q ₊	State
	X	X	Q	No Change
	0	0	Q	No Change
	0	1	0	Reset
	1	0	1	Set
	1	1	-	Invalid



Wave forms



Excitation Table

Q	Q ₊	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

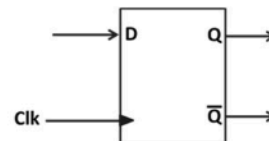
173

173

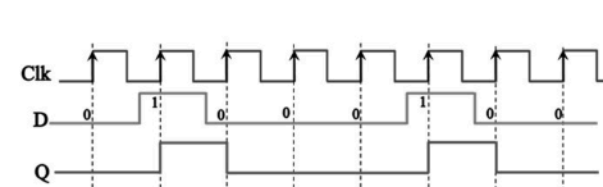
D Flip-flop

Truth Table

Clk	D	Q ₊	State
	X	Q	No Change
	0	0	Reset
	1	1	Set



Wave forms



Excitation Table

Q	Q ₊	D
0	0	0
0	1	1
1	0	0
1	1	1

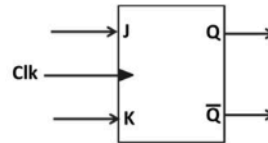
174

174

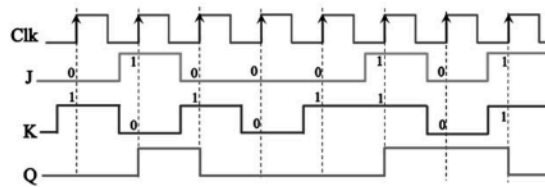
JK Flip-flop

Truth Table

Clk	J	K	Q_+	State
	X	X	Q	No Change
	0	0	Q	No Change
	0	1	0	Reset
	1	0	1	Set
	1	1	Q'	Toggle



Wave forms



Excitation Table

Q	Q_+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

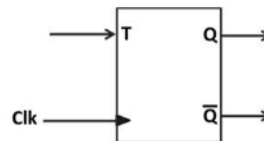
175

175

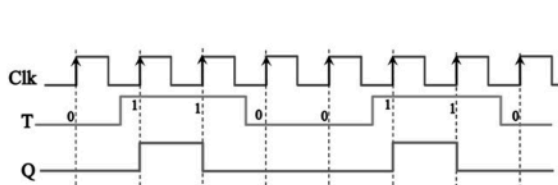
T Flip-flop

Truth Table

Clk	T	Q_+	State
	X	X	No Change
	0	Q	No Change
	1	Q'	Toggle



Wave forms



Excitation Table

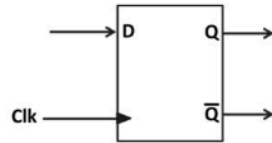
Q	Q_+	T
0	0	0
0	1	1
1	0	1
1	1	0

176

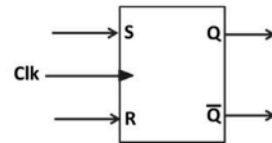
176

Symbols of flipflops

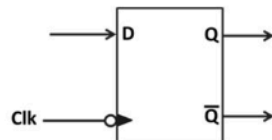
Positive Edge D flipflop



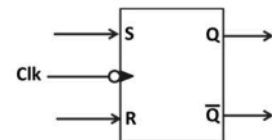
Positive Edge RS flipflop



Negative Edge D flipflop



Negative Edge RS flipflop

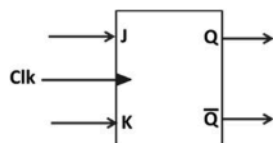


177

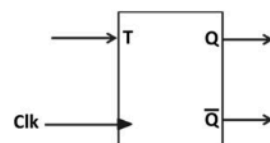
177

Symbols of flipflops

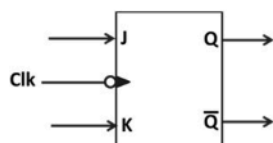
Positive Edge JK flipflop



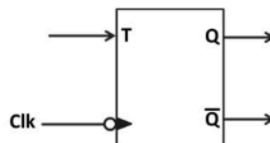
Positive Edge T flipflop



Negative Edge JK flipflop



Negative Edge T flipflop

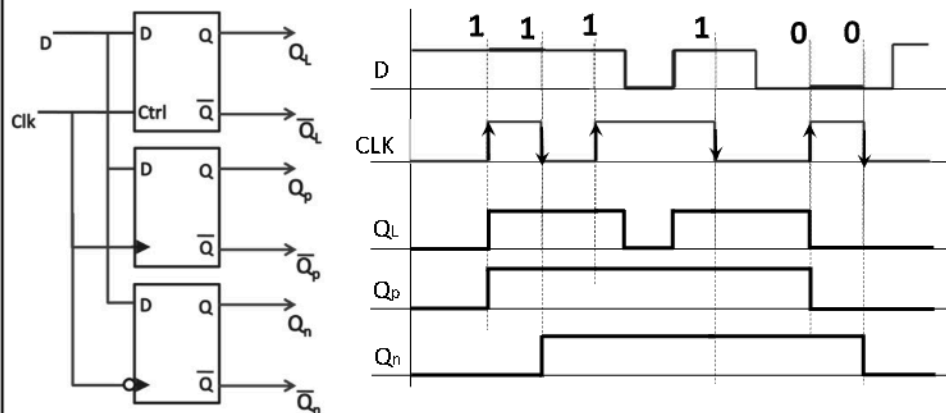


178

178

Timing Waveform

Draw the timing waveform for the below components, which receives same Clk and D input signals.



179

179

Flipflops with Asynchronous inputs.

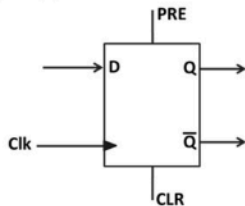
- ♦ The inputs of the flipflops like R, S, D, J, K & T are called synchronous inputs because, these inputs can bring a change in the outputs of the flipflops only at the instant of posedge or negedge of the clock.
- ♦ So, we need to wait till the instant of posedge or negedge of the clock to set or reset the flipflop.
- ♦ Hence, flipflops are provided with a pair of asynchronous inputs (preset & clear) to make the flipflop set or reset at any instant of time without waiting for the active edge of the clock signal.

180

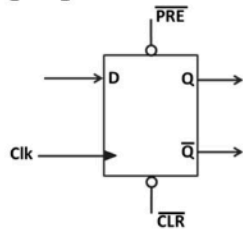
180

Flipflops with Asynchronous inputs.

Flipflop with active HIGH Asynchronous Inputs.



Flipflop with active LOW Asynchronous Inputs.



181

181

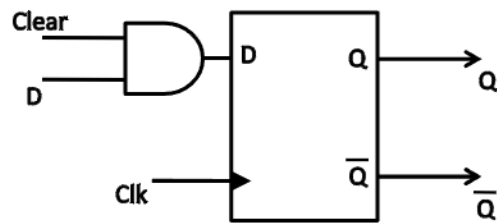
Active HIGH Asynchronous inputs.

- ♦ When preset is HIGH, the flipflop will be driven into set state without waiting for the posedge of the flipflop.
- ♦ When clear is HIGH, the flipflop will be driven into reset state without waiting for the posedge of the flipflop.
- ♦ For normal operation both the asynchronous inputs must be connected to '0'

182

182

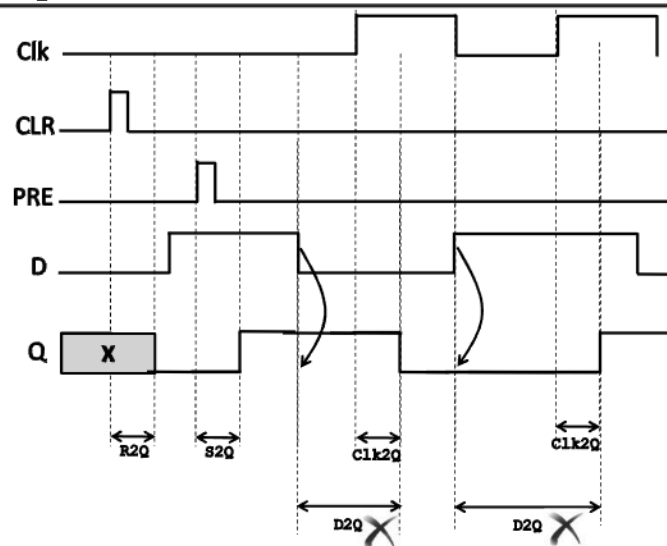
Synchronous Clear



183

183

Delays in Sequential Circuits



184

184

Delays in Sequential Circuits

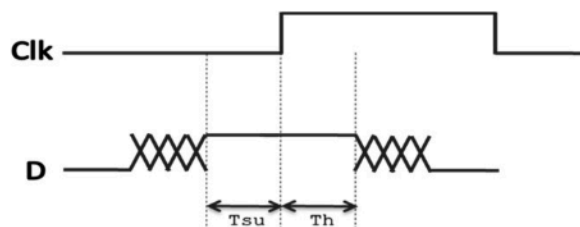
- ♦ **D2Q Delay** : change in input D alone cannot bring a change in the output . So, D2Q delay is not valid.
- ♦ **Clk2Q Delay** : Time taken by the flipflop to provide a stable output after a clock edge occurs.
- ♦ **R2Q Delay** : Time taken by flipflop to come to RESET state after the application of asynchronous CLEAR input.
- ♦ **S2Q Delay** : Time taken by flipflop to come to SET state after the application of asynchronous PRESET input.

185

185

Setup & Hold Times

- ♦ **Setup Time** : is the minimum time interval for which the input signal must be stable (unchanging) prior to the sampling event of the clock for the input signal to be recognized correctly.
- ♦ **Hold Time** : is the minimum time interval for which the input signal must be stable (unchanging) following the sampling event of the clock for the input signal to be recognized correctly.



186

186

Setup & Hold Times

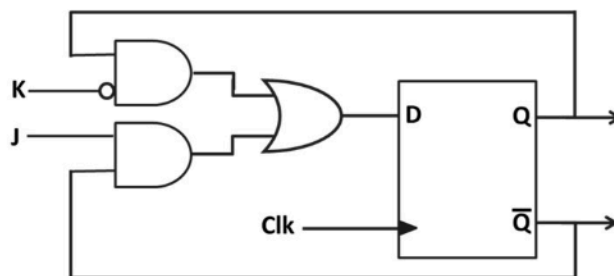
- ♦ A hold time violation is when a input signal change too quickly, after the clock's active transition
- ♦ A setup time violation is when a signal arrives too late, and misses the time when it should advance.
- ♦ If the setup or hold time parameters are violated, the output would be either logic0, logic1 or metastable state (output oscillates between 0 & 1).

Learn this in detail during
advanced course

187

187

FF Conversions



Learn this during advanced
course

$$D = J \overline{Q_n} + \overline{K} Q_n$$

188

188

Sequential Logic Circuits - II

189

189

Agenda

- ◆ Registers
- ◆ Counters

190

190

Registers

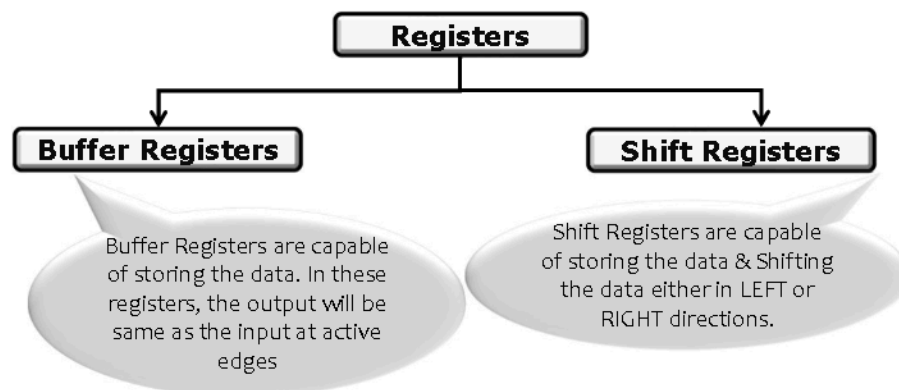
- ♦ Group of many flipflops to store multiple bits of information is called as “Register”.
- ♦ Registers can be used to perform the following two different functions.
 - ♦ Data storing
 - ♦ Data shifting

191

191

Classification of Registers

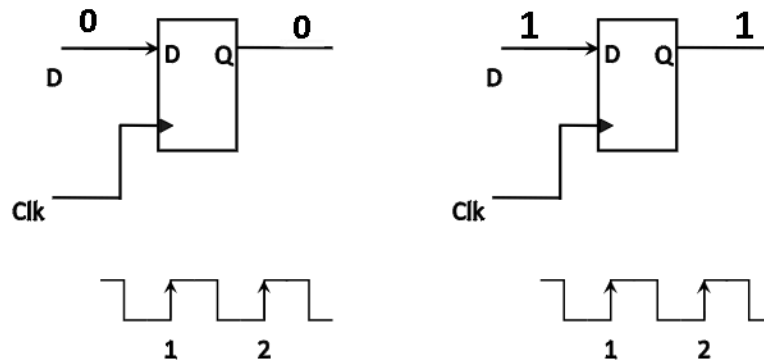
Registers can be classified as follows.



192

192

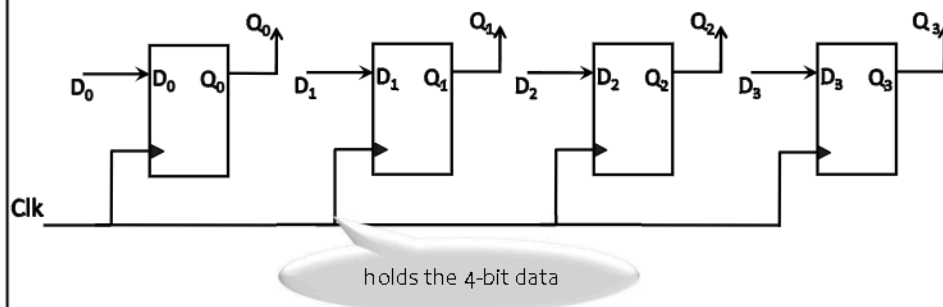
Functionality of a D flipflop



193

193

Buffer Register

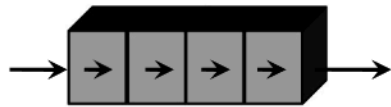


194

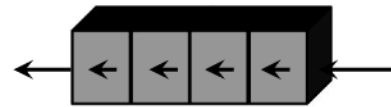
194

Serial Shift Registers

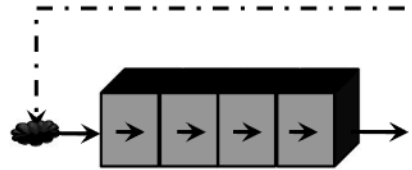
Serial In Serial Out (Right Shift)



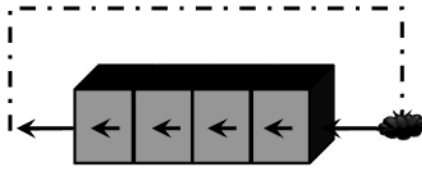
Serial In Serial Out (Left Shift)



Rotate Right



Rotate Left

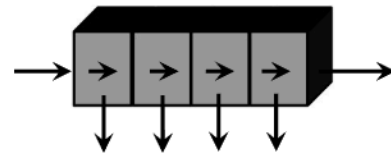


195

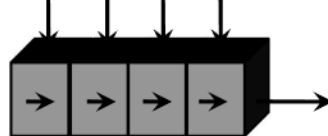
195

Parallel Shift Registers

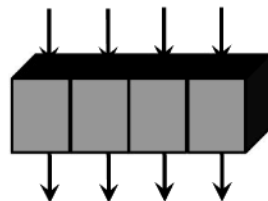
Serial In Parallel Out



Parallel In Serial Out



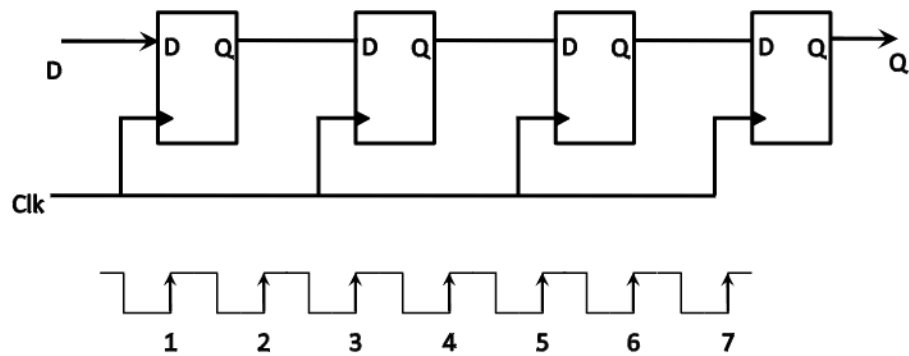
Parallel In Parallel Out



196

196

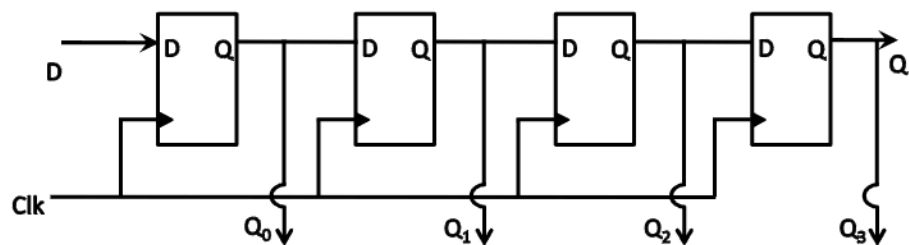
SISO Shift Registers



197

197

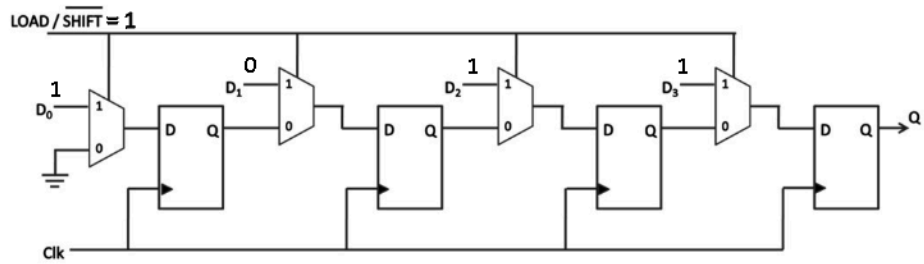
SIPO Shift Registers



198

198

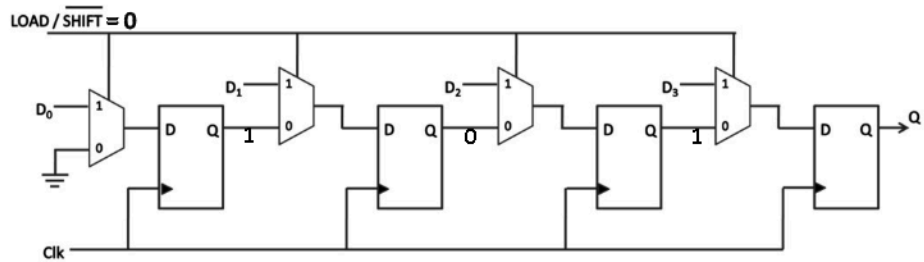
PISO Shift Register



199

199

PISO Shift Register

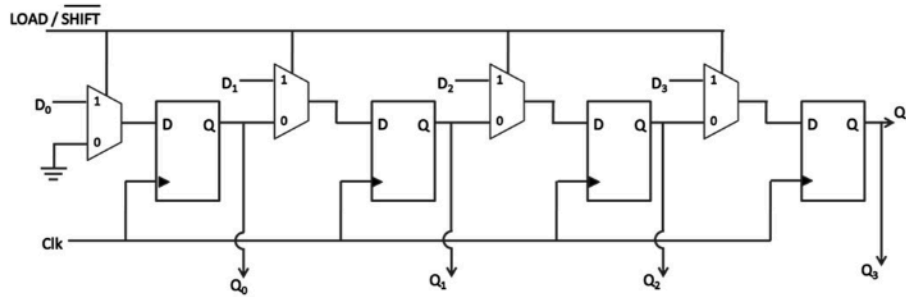


200

200

100

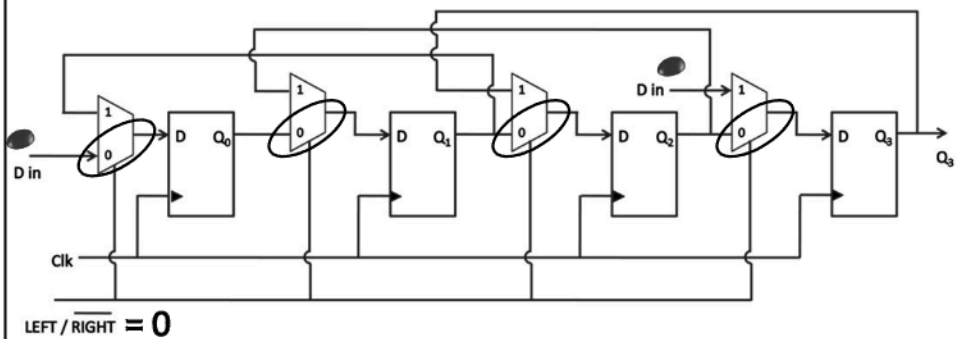
PIPO Shift Register



201

201

Bidirectional Shift Registers



202

202

101

[illegible]

203

Counters

What is a counter ?

- ♦ As the name suggests, it counts the no. of clock pulses.
- ♦ This counting can be done in a
 - ♦ Sequential manner to count the no. of clock pulses.
 - ♦ Random manner to generate a specific sequence.

204

What is a counter ?

- ◆ As the name suggests, it counts the no. of clock pulses.
- ◆ This counting can be done in a
 - ◆ Sequential manner to count the no. of clock pulses.
 - ◆ Random manner to generate a specific sequence.

204

Classification of Counters

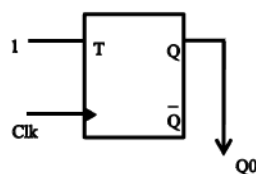
Counters can be classified based on

- ♦ **Clock** : Synchronous or Asynchronous
- ♦ **Clock Trigger** : Posedge trigger or Negedge trigger.
- ♦ **Counts** : Binary, Decade
- ♦ **Count Direction** : UP, DOWN, UP-DOWN, RANDOM
- ♦ **Flipflops Used** : JK or T or D.

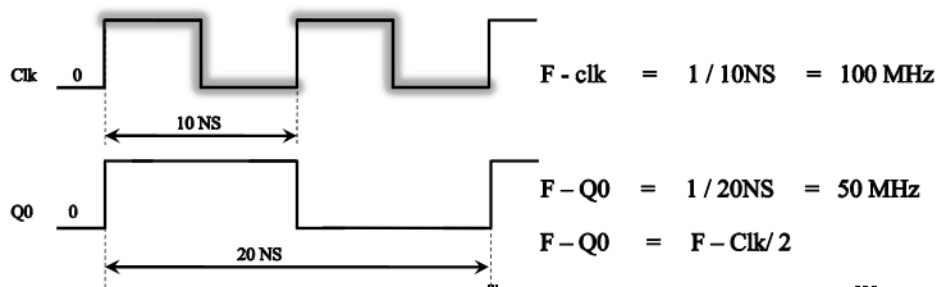
205

205

Concept of Frequency Division



- ♦ The output of a +ve edge triggered T-flipflop toggles at every +ve edge in the clock signal.
- ♦ And remains constant during the period other than the instant of +ve edge of the clock.

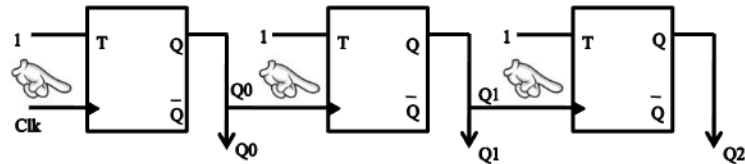


206

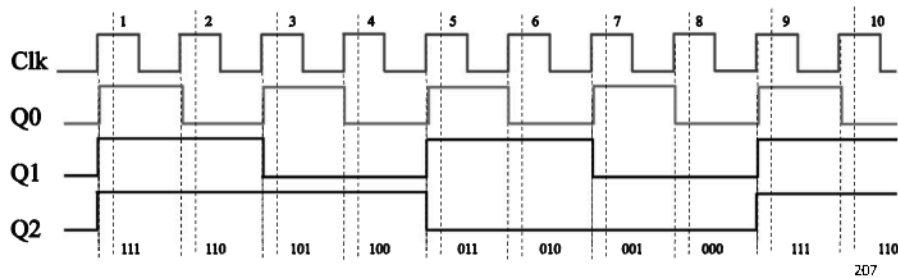
206

Asynchronous counter

3 Bit Asyn Down Counter using +ve edge triggered FF



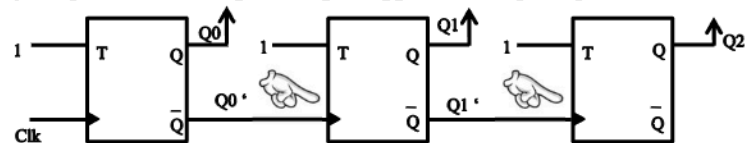
$$F(Q0) = F(\text{Clk}) / 2 \quad F(Q1) = F(Q0) / 2 \quad F(Q2) = F(Q1) / 2$$



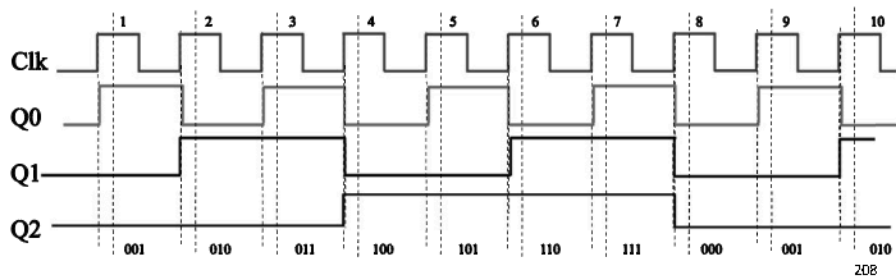
207

Asynchronous counter

3-bit Asyn. up counter using Posedge triggered T-Flip Flop

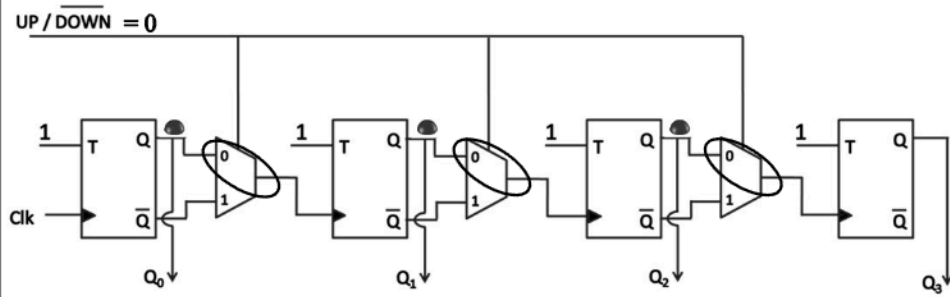


$$F(Q0) = F(\text{Clk}) / 2 \quad F(Q1) = F(Q0) / 2 \quad F(Q2) = F(Q1) / 2$$



208

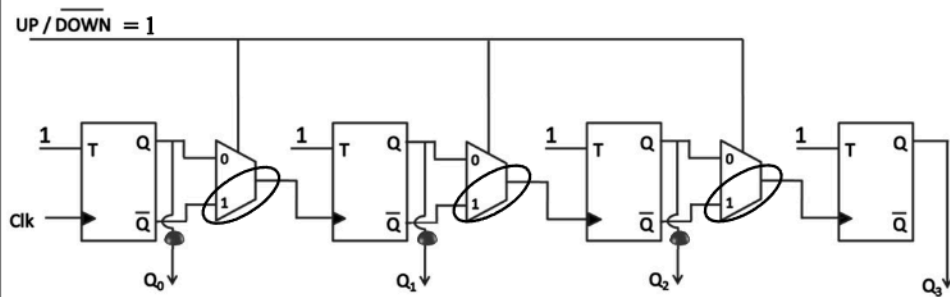
UP/DOWN Counter



209

209

UP/DOWN Counter

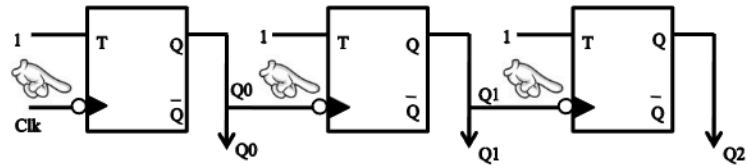


210

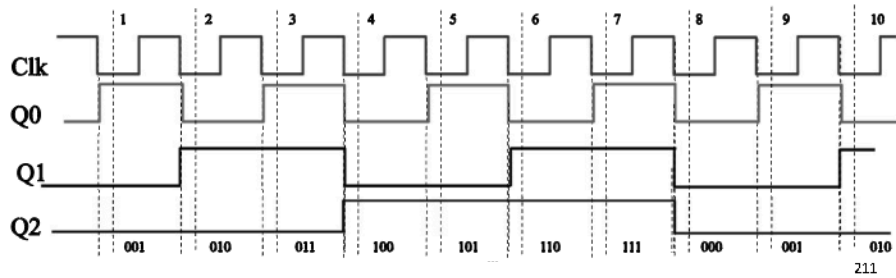
210

Asynchronous counter

3-bit Asyn. up counter using Negedge triggered T-Flip Flop



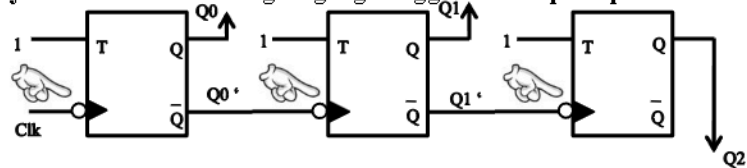
$$F(Q0) = F(\text{Clk}) / 2 \quad F(Q1) = F(Q0) / 2 \quad F(Q2) = F(Q1) / 2$$



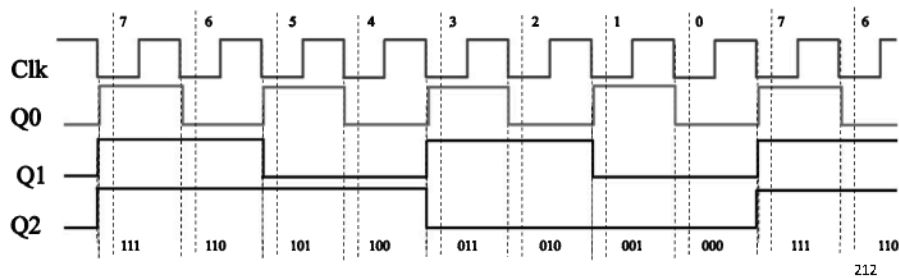
211

Asynchronous counter

3-bit Asyn. down counter using Negedge triggered T-Flip Flop



$$F(Q0) = F(\text{Clk}) / 2 \quad F(Q1) = F(Q0) / 2 \quad F(Q2) = F(Q1) / 2$$



212

Synchronous UP Counter

UP Counting Sequence

Q_2	Q_1	Q_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1
0	0	0

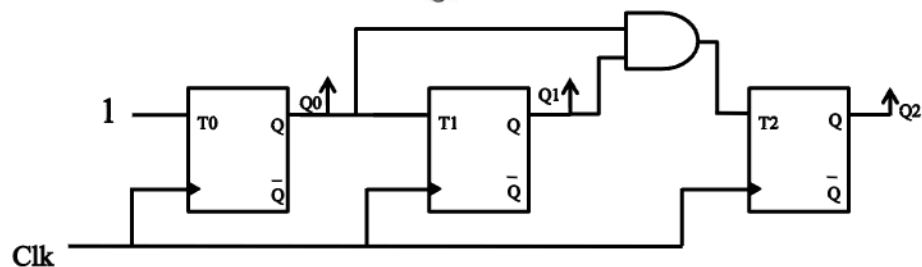
- ♦ Q_0 toggles for every cycle.
- ♦ And Q_1 toggles only when Q_0 is '1'
- ♦ Q_2 toggles only if (Q_1 & Q_0) are '1'.
- ♦ **Important Point :** T FF toggles only when the T input is '1'

213

213

Synchronous UP Counter

- ♦ Q_0 toggles for every cycle.  ♦ So, T_0 should be supplied with '1'.
- ♦ Q_1 toggles when Q_0 is '1'  ♦ T_1 should be supplied with Q_0 .
- ♦ Q_2 toggles only if (Q_1 & Q_0) '1'  ♦ T_2 should be supplied with Q_0 & Q_1



214

214

107



215



216

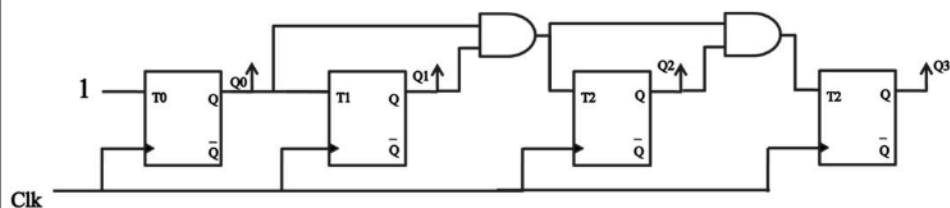
Modulus of a Counter

- ♦ The number of unique states that a counter may have before the sequence repeats itself is the modulus of the counter.
- ♦ An UP counter with 3 flipflops counts from 0 to 7 and again repeats from 0 after 7. So, the no. of states is 8 (i.e., 0 to 7).
- ♦ So, a counter with 'n' no. of flipflops counts from 0 to 2^n-1 . So, the Modulus of this counter is 2^n .
- ♦ The output frequency of the last FF of any counter will be the clock frequency divided by the MOD of the counter.
- ♦ A T flipflop with T input tied to Vcc acts as divide by 2 counter.

217

217

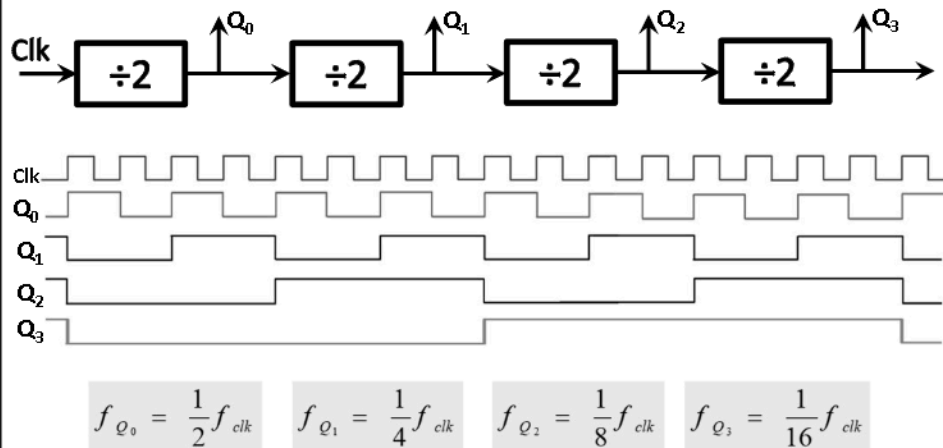
Mod-16 Counter



218

218

Frequency Division using Counters



219

219

Advantages of Syn. Counters over Asyn. Counters.

- ♦ The outputs of all the flipflops will change simultaneously.
- ♦ A synchronous counter can operate at much higher frequency, but the circuitry are more complex than that of the asynchronous counter.

220

220

110

Counter Design

- ♦ **Step 1:** Derive the State Transition Diagram
- ♦ **Step 2:** Draw State Transition Table
- ♦ **Step 3:** Choose flip-flop type for implementation

Use Excitation Table to remap Next State Functions

- ♦ **Step 4:** Obtain the expressions for input variables of the flipflops in terms of the present state values.
- ♦ **Step 5:** Minimize them using K-maps
- ♦ **Step 6:** Implement Resulting Logic with the minimised expressions.

You will learn during the advanced course

221

221

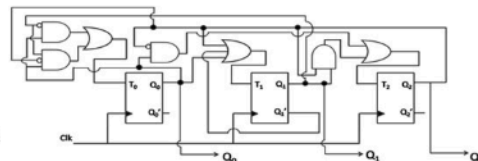
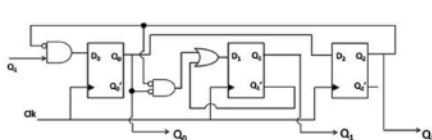
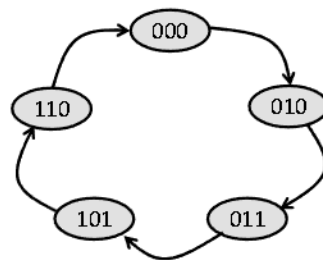
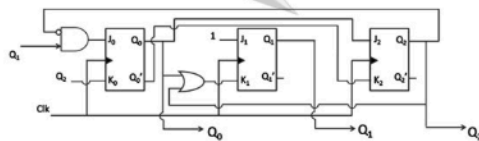
Counter Design

Design a counter that generates the sequence.

Example :

0,2,3,5,6,0,---

Counter using different ffs are shown here



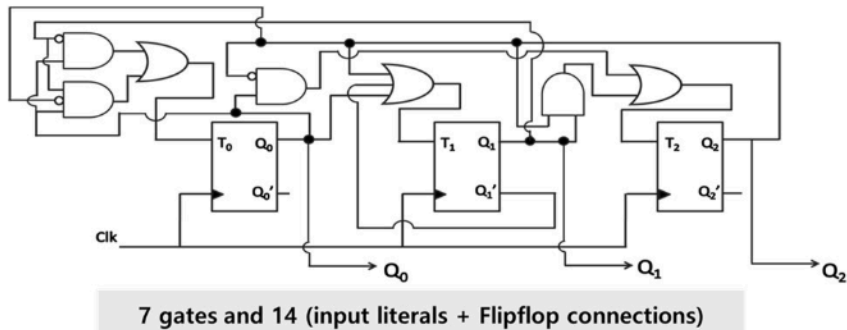
222

222

111

Comparison

- ♦ T FFs well suited for straightforward binary counters
- ♦ But yielded worst gate and literal count for this example

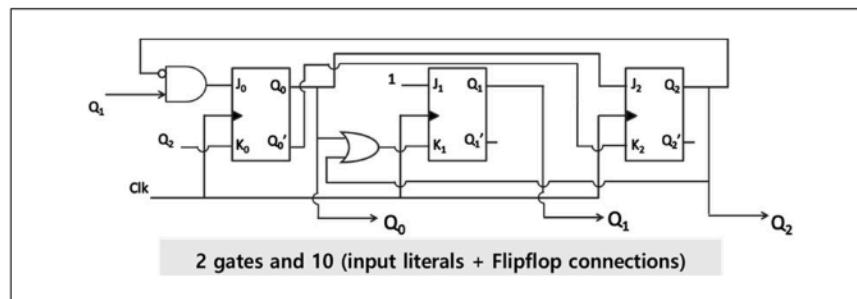


223

223

Comparison

- ♦ JK FFs yielded lowest gate count
- ♦ Tend to yield best choice for packaged logic where gate count is key



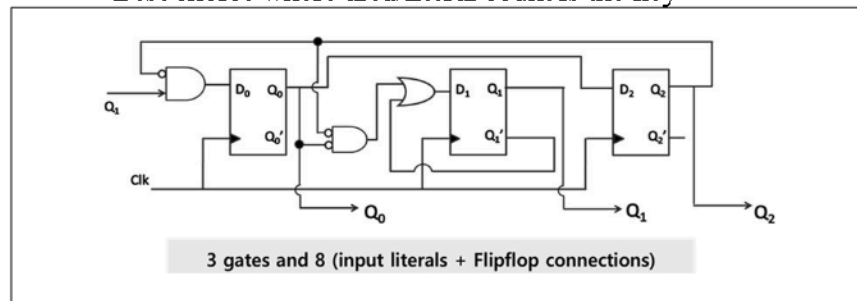
224

224

Comparison

- ◆ D FFs yield simplest design procedure

- ◆ Best literal count
- ◆ D storage devices very much transistor efficient in VLSI
- ◆ Best choice where area/literal count is the key



225

Shift Register Counter

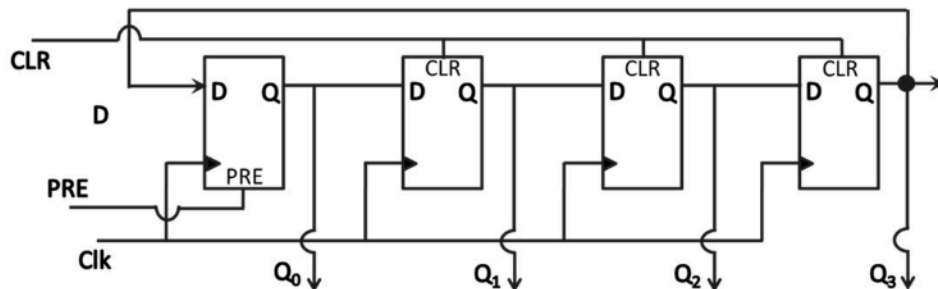
- ◆ We are aware of the fact that counters are used for producing a special sequence of numbers.
- ◆ Similarly, a shift register with the serial output connected back to the serial input to produce special sequence.
- ◆ Two of most common type of shift register counters are :
 - ◆ Ring counter
 - ◆ Johnson counter [Shift Counter / twisted ring counters].

226

226

113

Ring Counter



227

227

Ring Counter

Cycle Number	Q_0	Q_1	Q_2	Q_3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0
5	0	1	0	0
6	0	0	1	0
7	0	0	0	1

4 Cycles

4 Cycles

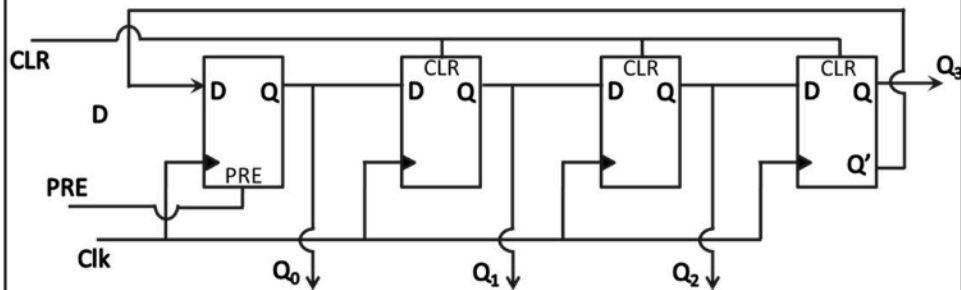
Modulus of a 'N' bit ring counter is 'N'.

For every 4 clock cycles, this counter goes into the same set of states. So, modulus of a 4 bit ring counter is 4.

228

228

Twisted Ring Counter



229

229

Twisted Ring Counter

Cycle Number	Q_0	Q_1	Q_2	Q_3	Q_3'
0	1	0	0	0	1
1	1	1	0	0	1
2	1	1	1	0	1
3	1	1	1	1	0
4	0	1	1	1	0
5	0	0	1	1	0
6	0	0	0	1	0
7	0	0	0	0	1
8	1	0	0	0	1

For every 8 clock cycles, this counter goes into the same set of states. So, modulus of a 4 bit Johnson counter is 8.

So, modulus of a 'N' bit Johnson counter is '2N'.

230

230

115

Finite State Machines

231

231

Agenda

- ◆ Finite State Machine
- ◆ Mealy Machine
- ◆ Moore Machine
- ◆ Design Examples
- ◆ Glitches and Hazards

232

232

116

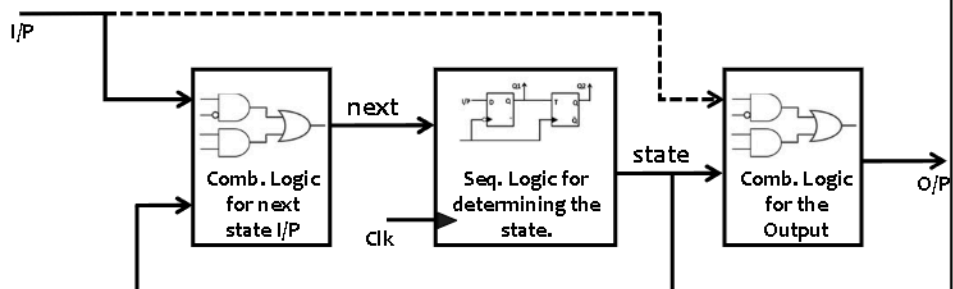
Finite State Machines

- ♦ Systematic way of specifying any sequential logic
- ♦ Ideally suited for complex sequential logic.
- ♦ To specify the sequential circuits which are :
 - ♦ complex in their transition sequence &
 - ♦ depend on several control inputs
- ♦ Different models in FSM are :
 - ♦ Mealy Model
 - ♦ Moore Model

233

233

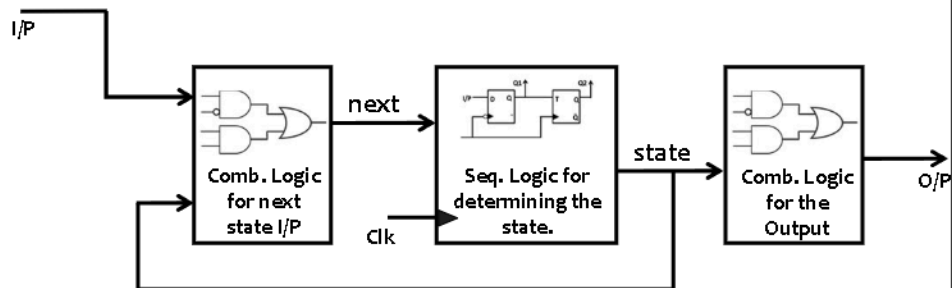
Mealy Machines



234

234

Moore Machines



235

235

Mealy Vs Moore

Mealy

- ♦ Output depends on both input & current state of the circuit.
- ♦ Next state is dependent on both input & current state.
- ♦ Outputs are Asynchronous. So, glitches can occur.
- ♦ Less no. of states. Obviously, less no. of flipflops are sufficient for implementation.
- ♦ Not safe to use.
- ♦ Area efficient.

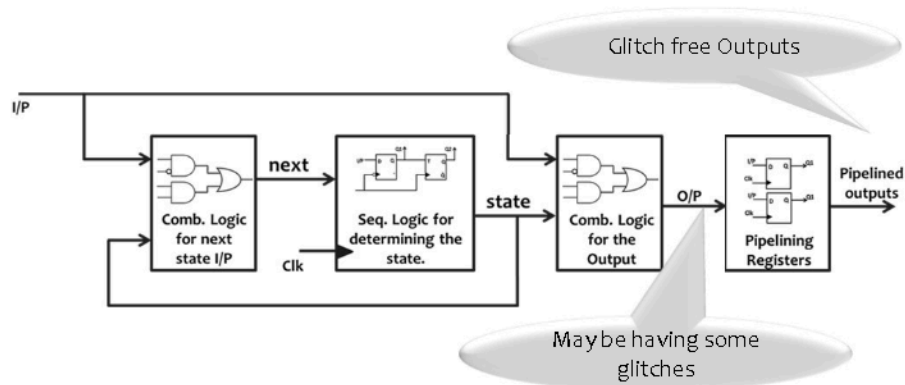
Moore

- ♦ Output depends only on current state of the circuit.
- ♦ Next state is dependent on both input & current state.
- ♦ Outputs are Synchronous. So, glitches can never occur.
- ♦ More no. of states. Obviously, more no. of flipflops are required for implementation.
- ♦ Safe to use.
- ♦ Not area efficient.

236

236

Mealy FSM



237

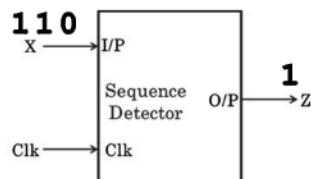
237

Sequence Detector - FSM Example

Specification

Design a sequence detector that receives binary data stream at its input, **X** and signals when a combination '110' arrives at the input by making its output, **Z** high which otherwise remains low.

Consider the first received bit is LSB



238

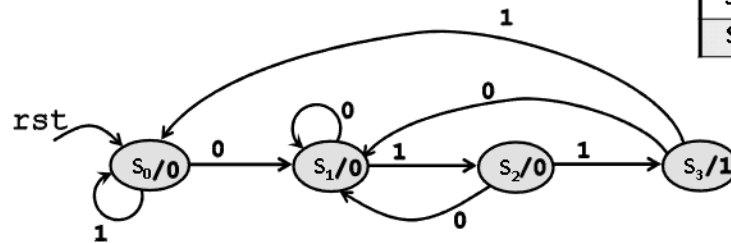
238

119

Sequence Detector - Moore

Sequence : 110

State	Meaning
S_0	Initial state
S_1	0
S_2	10
S_3	110

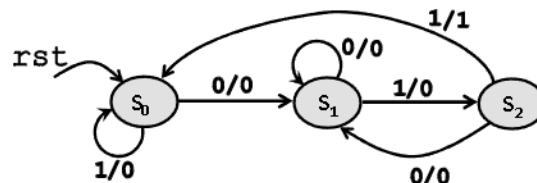


239

Sequence Detector - Mealy

Sequence : 110

State	Meaning
S_0	Initial state
S_1	0
S_2	10

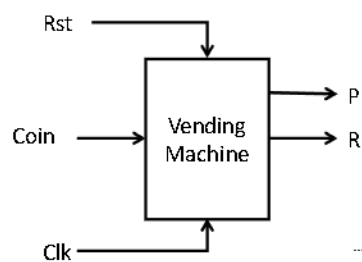


240

Vending Machine - FSM Design

Specification

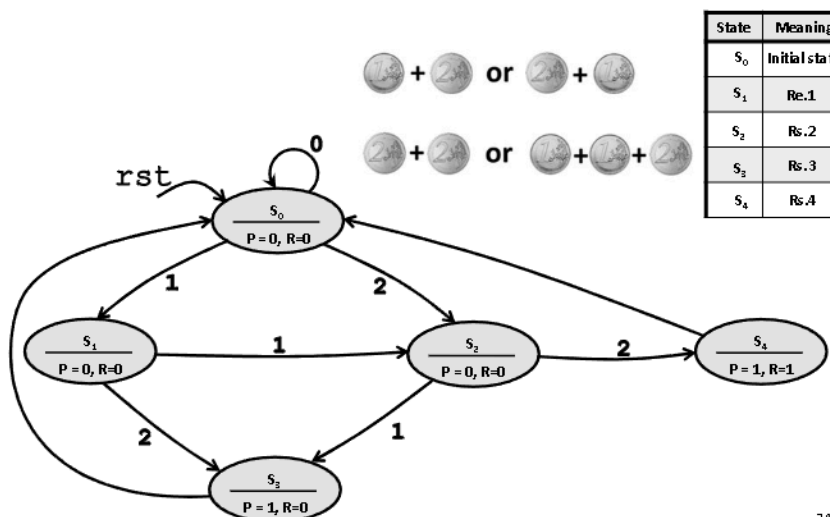
- Design a vending machine which delivers a product when it receives Rs.3/- and also it has to return the change if receives Rs.4/- . The machine can take only two types of coins of denomination 1 and 2 in any order.



243

243

Vending Machine



244

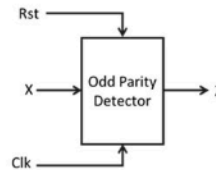
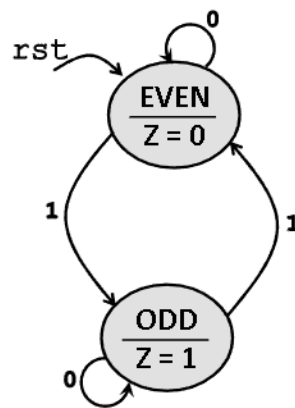
244

122

Odd Parity Detector-FSM Design

Specification

Assert output whenever input bit stream has odd number of 1's.



Present State (Q)	I/P (X)	Next State (Q+)	O/P (Z)
Even	0	Even	0
Even	1	Odd	0
Odd	0	Odd	1
Odd	1	Even	1

245

245

Odd Parity Detector-FSM Design

Specification

Assert output whenever input bit stream has odd number of 1's.

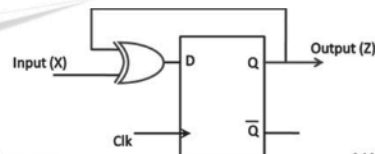
Present State (Q)	I/P (X)	Next State (Q+)	O/P (Z)
Even	0	Even	0
Even	1	Odd	0
Odd	0	Odd	1
Odd	1	Even	1

State Encoding

Design a circuit from FSM will be discussed in advanced course

$$\text{NextState} = \text{Din} = Q \oplus X$$

$$\text{Output (Z)} = Q$$



246

246

Glitches & Hazards

247

247

Glitches & Hazards

- ♦ When the input to a combinational circuit changes, unwanted switching transients (Glitches) may appear in the output.
- ♦ These transients occur when different paths from input to output have different propagation delays.
- ♦ A hazard is said to exist when a circuit has the possibility of producing such a glitch.

248

248

124

Types of Hazards

Static 1 - Hazard

- ♦ Possibility of a circuit's output producing a 0 glitch when we would expect the output to remain at a steady 1.



Static 0 - Hazard

- ♦ Possibility of a circuit's output producing a 1 glitch when we would expect the output to remain at a steady 0.



Dynamic Hazard

- ♦ Possibility of the output of a circuit changing more number of times when it should change from (0 to 1) or (1 to 0).



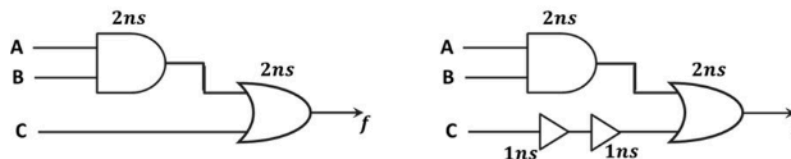
249

249

Elimination of Hazards

Technique 1

- ♦ Adding delay to balance the delays in the parallel paths can eliminate hazards since, unequal delays is the main cause of these glitches.



250

250

125

Elimination of Hazards

Technique 2

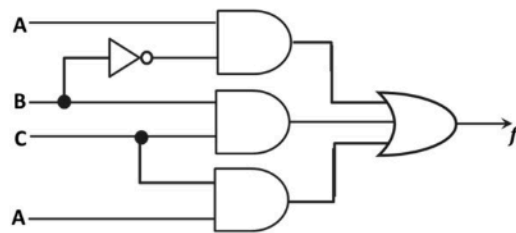
- Adding the redundant gates can eliminate the hazards.

$$f = \sum m(3,4,5,7)$$

		BC			
A		00	01	11	10
	0	0	0	1	0
	1	1	1	1	0

$$f = AB' + BC + AC$$

$$f = A\bar{B} + BC + AC$$



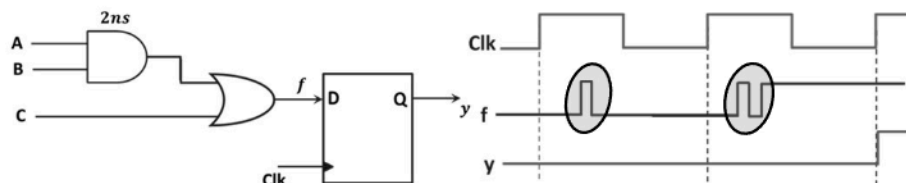
251

251

Elimination of Hazards

Technique 3

- Adding the pipelining registers at the outputs the combinational circuits.



252

252

Memories

253

253

Agenda

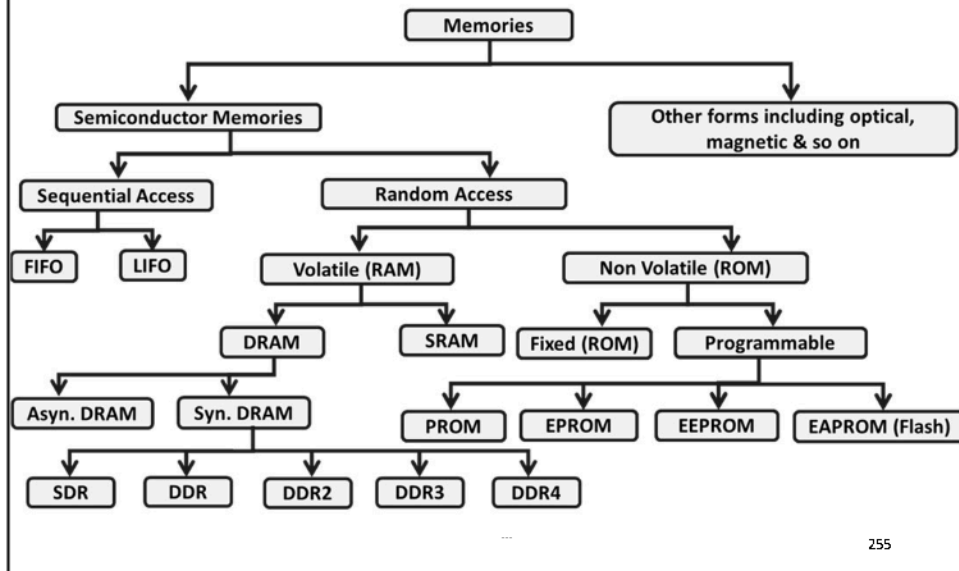
- ◆ Classification of Memories
- ◆ Basics of Memories
- ◆ Composition of Memories

254

254

127

Classification of Memories



255

Classification of Memories

- ♦ Optical memories like CD, DVD and magnetic memories like floppy, HDD will come under other forms of memories.
- ♦ Let's concentrate more on semiconductor memories.
- ♦ **Sequential Access Memories** : In these memories, data cannot be accessed from random address locations. It can be accessed in a serial manner.
- ♦ **Random Access Memories** : In these memories, data can be accessed from random address locations.
- ♦ FIFO & LIFO comes under sequential access memories.
- ♦ **Volatile Memories** : In these memories, stored data will be lost, if there is a loss in power supply.

256

256

128

Classification of Memories

- ♦ **Non - Volatile Memories** : In these memories, stored data will not be lost, even if there is a loss in power supply.
- ♦ SRAM & DRAM will come under volatile random access memories.
- ♦ **Synchronous DRAM** : In these memories, the basic read & write will be done in synchronism with a clock signal.
- ♦ **Asynchronous DRAM** : In these memories, the basic read & write will not be done in synchronism with a clock signal.
- ♦ **Single Data Rate**: 1 Byte of data for each clock cycle, either at posedge or negedge.
- ♦ **Double Data Rate** : 2 Bytes of data for each clock cycle. One data at posedge & second data at negedge.

257

257

Classification of Memories

- ♦ **Double Data Rate – 2nd Gen** : 4 Bytes of data for each clock cycle. two bytes at posedge & two more bytes at negedge.
- ♦ **Double Data Rate – 3rd Gen** : 8 Bytes of data for each clock cycle. four bytes at posedge & four more bytes at negedge.
- ♦ **Double Data Rate – 4th Gen** : 16 Bytes of data for each clock cycle. eight bytes at posedge & eight more bytes at negedge.
- ♦ In these days, DDR4 is the commonly used memory in LAPTOP's & Servers.
- ♦ **ROM**: Data stored in these memories is fixed at the time of fabricating the memories and cannot be changed again.
- ♦ No further write operation is possible. Only READ operation is possible.
- ♦ **Programmable ROM**: These memory chips can be programmed only once.

258

258

129

Classification of Memories

- ♦ **Erasable Programmable ROM** : These memory chips can be programmed for any no. of times & data will be erased by the application of UV light rays.
- ♦ Data stored in these memories can be programmed by the user based on his requirement. Both read & write are possible in these memories.
- ♦ **Electrically Erasable Programmable ROM**. These memory chips can be programmed for any no. of times & data will be erased by the application of electrical signals.
- ♦ **Erasable Alterable Programmable ROM**. These memories are special type of EEPROMs and data can be written at the block level instead of byte level.
- ♦ Can be programmed while in the circuit. USB drives, Memory cards & SSD's will come under this category.

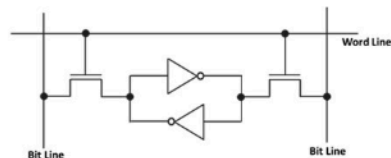
259

259

SRAM vs DRAM

Static RAM

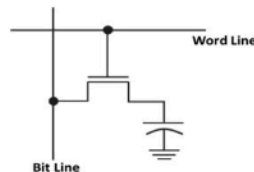
- ♦ Static RAM uses static devices like transistors & MOSFET's to store the data.



- ♦ At least, 6 no. of transistors are required for implementing one bit memory cell.
- ♦ Not area efficient. Occupies more area.
- ♦ No periodic refresh required.
- ♦ Less access time b/c of static devices.
- ♦ Standard IC fabrication process is sufficient for fabricating these memories.

Dynamic RAM

- ♦ Dynamic RAM uses dynamic devices like capacitors to store the data.



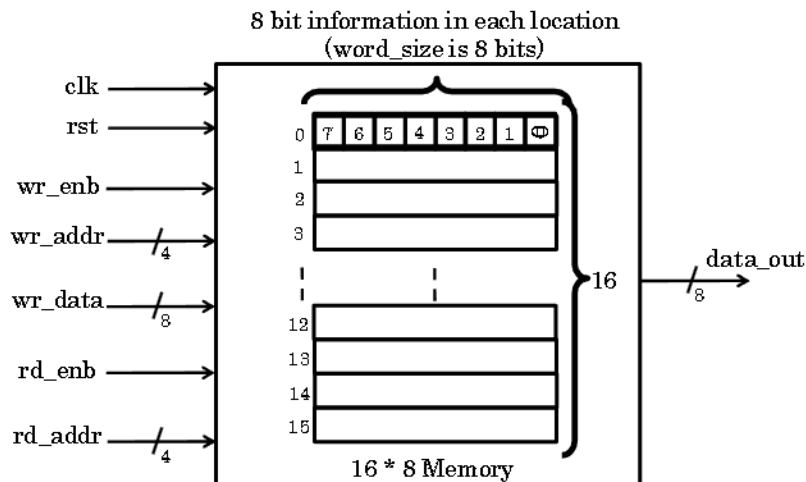
- ♦ One transistor & one capacitor are sufficient for implementing one bit memory.
- ♦ Area efficient. Occupies less area.
- ♦ Periodic refresh required.
- ♦ High access time b/c of usage of capacitors.
- ♦ Special IC fabrication process is required for fabricating these memories.

260

260

130

Basic Concepts of Memories



261

261

Composition of Memories

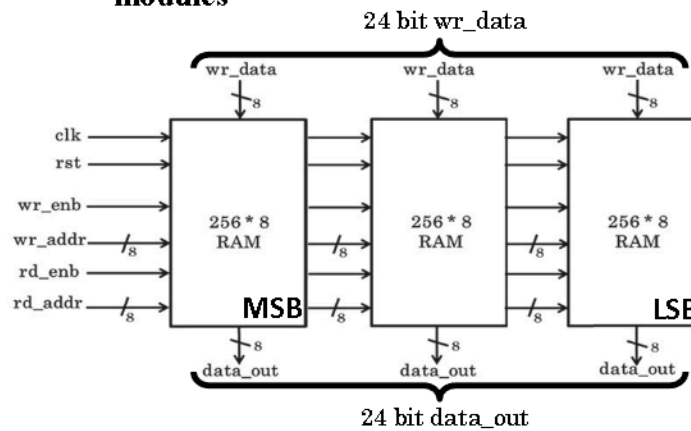
- ♦ It is the process of creating large memory module using smaller memory module.
- ♦ The following are the different requirements.
 - ♦ We may need to construct a large memory with larger word size (more bits per word) or
 - ♦ We may need to construct large memory with Wider addressing space (more words) or
 - ♦ Both

262

262

Memory with larger word size

Example : Construct 256*24 RAM using 256*8 RAM modules



263

263

Memory with more words

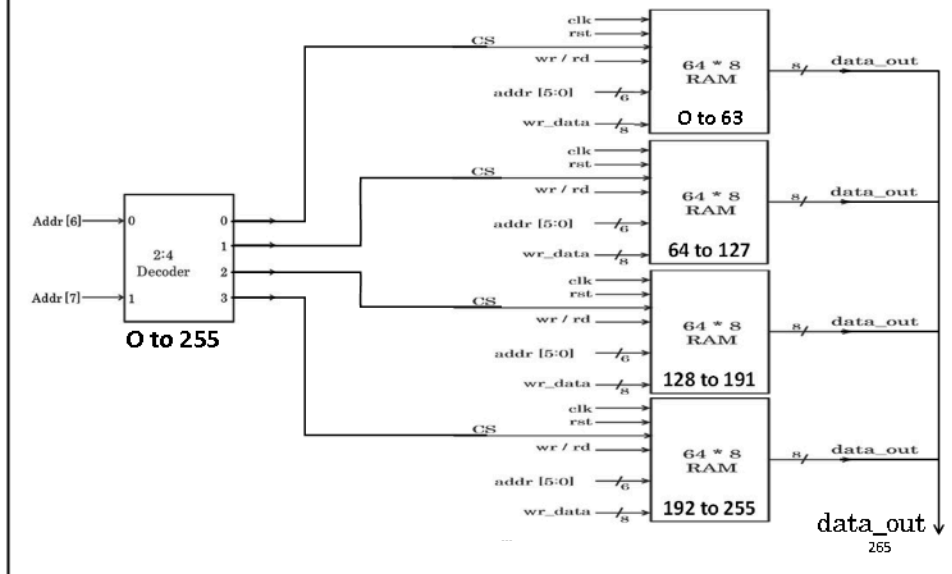
Example : Construct 256*8 RAM using 64*8 RAM modules

- ♦ For constructing 256 * 8 Memory using 64 * 8 Memories, we need 4 modules of 64 * 8 memories since $256/64 = 4$.
- ♦ The available 64 * 8 Memory will have 6 address lines.
- ♦ But, the required memory should have 8 address lines.
- ♦ The extra 2 lines can be used for selecting any of the 4 smaller memory modules by using a decoder logic.
- ♦ Recall the fact that, only one of the output line of a decoder will be at HIGH level depending on the input available.

264

264

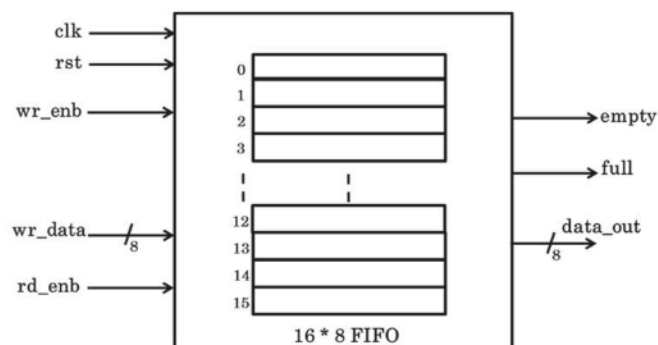
Memory with more words



265

FIFO

- ♦ Used to pass data between two asynchronous clock domains.
- ♦ Used to pass data between two systems working with same clock but with different throughput.



266

133

266

Cache Memory

Speed



Data Storage Capacity



267

267

Cache Memory

- ♦ CPU Contains Registers and Cache Memory So, that it can access frequently accessed data at a faster speed instead of accessing from slower memories like RAM & HDD.
- ♦ CPU has only few MB of Cache. But, CPU makes a tremendous use of this small memory because the speed of data access in cache is high because it need not do periodic refresh unlike DRAM.
- ♦ Cache uses SRAM Technology. So, Many no. of memory cells cannot be fabricated in a small processor. So, cache memories will be in the order of MB unlike DRAM which will be in the order of GB.
- ♦ For a better performance, Use a processor with more Cache Memory.

268

268

